

ScriptumAI

Pedro Seara, João Meira, Leonardo Vieira
(nr^{os} 23079, 23505, 34688)

Professora Patrícia Leite

Professor Paulo Sousa

Professor Nuno Mendes

PROJETO MULTIMODAL (PMM)

PROGRAMAÇÃO WEB AVANÇADA (PWA)

PROGRAMAÇÃO AVANÇADA DE DISPOSITIVOS MÓVEIS (PADM)

MESTRADO EM ENGENHARIA INFORMÁTICA

ESCOLA SUPERIOR DE TECNOLOGIA

INSTITUTO POLITÉCNICO DO CÁVADO E DO AVE

Identificação dos alunos

Pedro Seara, João Meira, Leonardo Vieira

Alunos número 23079, 23505, 34688

Mestrado em Engenharia Informática

Projeto Multimodal (PMM)

Programação Web Avançada (PWA)

Programação Avançada de Dispositivos Móveis (PADM)

Professora Patrícia Leite

Professor Paulo Sousa

Professor Nuno Mendes

Conteúdo

1	Introdução	1
1.1	Motivação	1
1.2	Descrição	2
2	Modelo de Negócio	3
2.1	Objetivos	3
2.2	Público-Alvo	3
2.3	Resultados Esperados	4
2.4	Utilizadores	4
2.5	Proposta de Valor	4
2.6	Canais	5
2.7	Relacionamento com Clientes	5
2.8	Fontes de Receita	5
2.9	Recursos-Chave	6
2.10	Atividades-Chave	6
2.11	Parcerias-Chave	7
2.12	Estrutura de Custos	7
3	Requisitos do Cliente	9
3.1	Requisitos Funcionais	9
3.1.1	RF1:Gestão de Utilizadores e Autenticação	9
3.1.2	RF2:Gestão Documental	10
3.1.3	RF3:Assistente Conversacional (RAG Chatbot)	11
3.1.4	RF4:Multi-tenancy e Isolamento de Dados	11
3.1.5	RF5:Integrações Externas	12
3.1.6	RF6:Gestão de Departamentos	12
3.2	Requisitos Não Funcionais	12
3.2.1	RNF1:Desempenho	12

3.2.2	RNF2:Escalabilidade	12
3.2.3	RNF3:Segurança	13
3.2.4	RNF4:Disponibilidade	13
3.2.5	RNF5:Manutenibilidade	13
3.2.6	RNF6:Observabilidade	14
3.2.7	RNF7:Cópias de Segurança (<i>Backups</i>)	14
3.3	Requisitos de <i>Hardware</i> e <i>Software</i>	14
3.3.1	Requisitos de <i>Hardware</i> (Servidor / Infraestrutura)	14
3.3.2	Requisitos de <i>Hardware</i> (Cliente / Utilizador Final)	15
3.3.3	Requisitos de <i>Software</i> (Servidor)	16
3.3.4	Requisitos de <i>Software</i> (Cliente)	16
4	Wireframes da Aplicação Móvel	17
4.1	Fluxo de Autenticação	17
4.2	Gestão de Documentos	18
4.3	Definições e Gestão de Perfil	19
4.4	Gestão de Utilizadores (Administração)	20
4.5	Notificações	21
4.6	Mockups de Alta Fidelidade	22
4.6.1	Fluxo de Autenticação	22
4.6.2	Ecrã Principal e Gestão de Documentos	23
4.6.3	Perfil, Definições e Notificações	24
4.6.4	Gestão de Utilizadores (Administração)	25
5	Especificação	27
5.1	Modelo de Domínio	27
5.1.1	Entidade: Organization	27
5.1.2	Entidade: User	28
5.1.3	Entidade: Document	28
5.1.4	Entidade: Invite	29
5.1.5	Entidade: Department	29
5.1.6	Entidade: TokenBlacklist	29
5.1.7	Relações Lógicas entre Entidades	30
5.2	Diagrama de Classes	30
5.2.1	IDP Service	30
5.2.2	Organization Service	31

5.2.3	Document Service	31
5.2.4	Notification Service	32
5.3	Diagramas de Casos de Uso	32
5.3.1	CU1: Login de Utilizador	32
5.3.2	CU2: Upload de Documento	34
5.3.3	CU3: Criação de Convite para Novo Utilizador	36
5.4	Diagramas de Sequência	38
5.4.1	Fluxo de Autenticação (Login)	38
5.4.2	Fluxo de Criação de Convite	39
5.4.3	Fluxo de Registo de Utilizador	41
6	Arquitectura Funcional	43
6.1	Visão Geral	43
6.2	Diagrama de Contexto (Nível 1)	44
6.3	Diagrama de <i>Contentores</i> (Nível 2)	45
6.3.1	Microserviços	46
6.3.2	Componentes Auxiliares (<i>Backing Services</i>)	47
6.4	Padrões de Comunicação	48
6.4.1	Comunicação Síncrona (REST/HTTP)	48
6.4.2	Comunicação Assíncrona (AMQP / RabbitMQ)	49
6.5	Arquitectura de Infraestrutura	50
7	Implementação da API	51
7.1	Visão Geral da Arquitectura	51
7.2	API Gateway	52
7.3	Serviço de Autenticação	52
7.4	Serviço de Organização	52
7.5	Serviço de Documentos	53
7.6	Serviço de IA	54
7.7	Serviço de Notificações	55
7.8	Comunicação entre Serviços	55
7.9	Armazenamento e Infraestrutura	56
8	Implementação da Aplicação Web	57
8.1	Tecnologias Utilizadas	57
8.2	Estrutura do Projeto	57
8.3	Autenticação e Gestão de Sessão	58

8.4	Funcionalidades Implementadas	58
8.4.1	<i>Dashboard</i>	58
8.4.2	Gestão de Membros	59
8.4.3	Gestão de Convites	59
8.4.4	Gestão de Departamentos	59
8.4.5	Secção de <i>Staff</i>	59
8.5	Comunicação com o <i>Backend</i>	60
8.6	Responsividade	60
9	Implementação da Aplicação Móvel	61
9.1	Tecnologias Utilizadas	61
9.2	Arquitectura	61
9.3	Camada de Rede	62
9.4	Casos de Uso	62
9.5	Funcionalidades Implementadas	62
9.5.1	Autenticação	62
9.5.2	Navegação Principal	63
9.5.3	Gestão de Documentos	63
9.5.4	Detalhe do Documento e Chatbot RAG	64
9.5.5	Assistente de IA Global	64
9.5.6	Notificações <i>Push</i>	64
9.5.7	Perfil e Suporte	64
9.6	Interface e Experiência do Utilizador	65
10	Propósitos e Objetivos Cumpridos	67
10.1	Análise de Cumprimento dos Objetivos	67
10.2	Funcionalidades Implementadas	68
10.2.1	Aplicação Web (PWA)	68
10.2.2	Aplicação Móvel (Android)	68
10.2.3	Servidor e Infraestrutura	69
10.3	Requisitos Avançados Implementados	69
11	Testes Realizados	71
11.1	Testes Unitários	71
11.1.1	Authentication Service	71
11.1.2	Document Service	72
11.2	Testes de Integração	72

11.2.1 Authentication Service	72
11.3 Testes <i>End-to-End</i>	72
12 Restrições	73
12.1 Limitações Funcionais	73
12.2 Segurança e Privacidade	73
13 Conclusão	75
13.1 Síntese	75
13.2 Resultados Principais	75
13.3 Limitações Observadas	76

Lista de Figuras

4.1	Wireframes do fluxo de Autenticação (Ecrã inicial, Login, Registo e Recuperação de <i>Password</i>).	18
4.2	Wireframes do fluxo de Gestão de Documentos (Listagem, Filtros, Adição e Detalhe).	18
4.3	Wireframes do fluxo de Definições e Gestão de Perfil.	19
4.4	Wireframes do fluxo de Gestão de Utilizadores (<i>Sidebar</i> , Lista de Utilizadores e Convites).	20
4.5	Wireframes do Centro de Notificações (Não Lidas, Todas e Estado Vazio).	21
4.6	Mockups do fluxo de autenticação.	22
4.7	Mockups do ecrã principal com filtros por departamento.	23
4.8	Mockups da gestão de documentos.	23
4.9	Mockups de perfil, definições e notificações.	24
4.10	Mockups da gestão de utilizadores (Administração).	25
5.1	Diagrama de classes do IDP Service	30
5.2	Diagrama de classes do Organization Service	31
5.3	Diagrama de classes do Document Service	31
5.4	Diagrama de classes do Notification Service	32
5.5	CU1 — Login: fluxo principal (sucesso)	33
5.6	CU1 — Login: credenciais inválidas	33
5.7	CU1 — Login: conta desativada	34
5.8	CU2 — Upload de Documento: fluxo principal (sucesso)	35
5.9	CU2 — Upload de Documento: ficheiro inválido	35
5.10	CU2 — Upload de Documento: erro no storage	36
5.11	CU3 — Criação de Convite: fluxo principal (sucesso)	37
5.12	CU3 — Criação de Convite: email já existe	37
5.13	CU3 — Criação de Convite: permissão negada	38
5.14	Diagrama de sequência do fluxo de autenticação	39
5.15	Diagrama de sequência do fluxo de criação de convite	40

5.16	Diagrama de sequência do fluxo de registo de utilizador	41
6.1	Diagrama de Contexto:Actores e Sistema ScriptumAI	44
6.2	Diagrama de <i>Contentores</i> :Microserviços e Componentes	45
6.3	Padrão de Comunicação Síncrona via REST/HTTP	48
6.4	Padrão de Comunicação Assíncrona via <i>Message Broker</i>	49
6.5	Arquitectura global de infraestrutura do sistema	50

Lista de Tabelas

3.1	Endpoints do módulo de gestão de departamentos	12
3.2	Requisitos de <i>hardware</i> da infraestrutura de produção	14
3.3	Requisitos de <i>hardware</i> para o serviço de IA (AI Service)	15
3.4	Requisitos de <i>software</i> do lado do servidor	16
3.5	Requisitos de <i>software</i> do lado do cliente	16
5.1	Relações lógicas entre microsserviços	30
10.1	Estado de cumprimento dos objetivos propostos	68
11.1	Suites de testes unitários do Authentication Service	71
11.2	Testes E2E do Organization Service	72

1. Introdução

O **ScriptumAI** é uma plataforma de processamento e gestão de documentos baseada em inteligência artificial. Concebida como solução *AIaaS* (*AI as a Service*), permite que uma única instância da aplicação sirva simultaneamente múltiplas organizações de qualquer sector de actividade, mantendo o isolamento total de dados entre cada *tenant*.

1.1 Motivação

As pequenas e médias empresas de qualquer sector enfrentam diariamente desafios crescentes na gestão eficiente dos seus documentos. O volume documental aumenta continuamente, os formatos são heterogéneos e a pesquisa manual é lenta e propensa a erros. As soluções existentes no mercado apresentam limitações relevantes:

- **Ferramentas genéricas:** não contemplam as especificidades sectoriais nem a terminologia própria de cada área de negócio;
- **Custo elevado:** a implementação de sistemas personalizados por empresa é economicamente inviável para organizações de pequena e média dimensão;
- **Isolamento de dados insuficiente:** soluções *multi-tenant* frequentemente não garantem separação rigorosa dos dados entre organizações distintas;
- **Ausência de automação inteligente:** a extração de informação relevante e a classificação automática de documentos raramente estão disponíveis em soluções acessíveis ao mercado PME.

O avanço dos *Large Language Models* (LLMs) e das técnicas de *Retrieval-Augmented Generation* (RAG) abre uma oportunidade concreta: disponibilizar capacidades de processamento e pesquisa documental por linguagem natural a um custo operacional partilhado, tornando estas funcionalidades acessíveis a qualquer organização, independentemente da sua dimensão.

Neste contexto, o **ScriptumAI** surge como resposta directa a este problema, oferecendo uma plataforma centralizada, segura e escalável, capaz de servir múltiplas organizações de forma simultânea e isolada, sem os custos de uma solução dedicada.

1.2 Descrição

O **ScriptumAI** é uma plataforma *SaaS* (*Software as a Service*) multi-tenant orientada para a gestão inteligente de documentos. A plataforma é independente de sector: qualquer organização pode utilizá-la independentemente da sua área de actividade, organizando os seus documentos pelos departamentos que ela própria define.

A plataforma oferece as seguintes capacidades principais:

Gestão documental Upload, organização, pesquisa e acesso a documentos com controlo de permissões por perfil de utilizador (Administrador, Gestor e Cliente). Suporta organização por departamentos e edição de metadados dos documentos existentes.

Processamento automático por IA Extração de texto, geração de resumos e identificação de palavras-chave através de modelos de inteligência artificial.

Assistente conversacional (*chatbot* RAG) Interface de linguagem natural que permite interrogar os documentos da organização, acessível por documento via API REST. O assistente utiliza *Retrieval-Augmented Generation*: gera *embeddings* semânticos com o modelo *paraphrase-multilingual-MiniLM-L12-v2* e recupera os excertos mais relevantes do documento para contextualizar as respostas do modelo Llama 3.1. O acesso é controlado por um sistema de quotas associado ao plano de subscrição do utilizador. O assistente responde apenas com base nos documentos do próprio *tenant*, garantindo isolamento total da informação.

Aplicação móvel Acesso completo às funcionalidades a partir de dispositivos Android, com interface desenhada segundo os princípios do *Material Design*.

Arquitectura multi-tenant Cada organização opera num espaço lógico isolado, identificado por subdomínio ou *header* HTTP, sem qualquer possibilidade de acesso cruzado entre *tenants*.

Segurança e auditoria Autenticação via JWT (*JSON Web Token*), controlo de acesso baseado em papéis (RBAC), registo de auditoria completo e comunicações cifradas por TLS.

A implementação baseia-se numa arquitectura de microsserviços orquestrada em Kubernetes, com *pipelines* CI/CD automatizadas e monitorização contínua através de Prometheus e Grafana, garantindo disponibilidade, escalabilidade e manutenibilidade em ambiente de produção.

2. Modelo de Negócio

O modelo de negócio foi estruturado segundo os blocos fundamentais de análise estratégica, permitindo compreender como a plataforma cria, entrega e captura valor para os seus clientes.

2.1 Objetivos

Os objetivos estratégicos da plataforma são:

- Disponibilizar uma plataforma de gestão documental com processamento inteligente por IA, acessível a pequenas e médias empresas de qualquer sector de actividade;
- Reduzir o tempo e custo associados à pesquisa, classificação e extração de informação em documentos organizacionais;
- Oferecer um modelo de subscrição *SaaS* com preços escaláveis, eliminando a necessidade de investimento inicial em infraestrutura por parte dos clientes;
- Garantir conformidade com requisitos regulamentares de privacidade e segurança de dados (RGPD), através de isolamento rigoroso entre *tenants*;
- Crescer para mais organizações e tipologias documentais, tirando partido da arquitectura multi-tenant genérica.

2.2 Público-Alvo

A plataforma destina-se a organizações de pequena e média dimensão de qualquer sector de actividade que necessitem de gerir, processar e pesquisar documentos de forma eficiente. A organização interna dos documentos é feita através de departamentos definidos pela própria organização, adaptando-se à sua estrutura real.

O decisor de compra é tipicamente o diretor de operações ou o responsável de TI da organização, motivado pela necessidade de digitalizar e automatizar processos documentais sem incorrer em custos de desenvolvimento à medida.

2.3 Resultados Esperados

A adopção da plataforma pelas organizações clientes visa produzir os seguintes resultados mensuráveis:

- Redução de até 60% no tempo despendido na pesquisa e localização de documentos;
- Eliminação de erros de classificação manual e perda de documentos;
- Capacidade de resposta mais rápida a pedidos internos e externos que requeiram consulta documental;
- Diminuição dos riscos de incumprimento regulamentar por falta de rastreabilidade documental;
- Democratização do acesso a ferramentas de IA aplicadas à gestão documental, anteriormente reservadas a grandes organizações.

2.4 Utilizadores

A plataforma define três perfis de utilizador com permissões diferenciadas:

Administrador Utilizador com acesso total à plataforma. Responsável pela configuração global da organização, gestão de todos os utilizadores, acesso a *logs* de auditoria e definição das integrações externas. Existe pelo menos um administrador por organização (*tenant*).

Gestor Utilizador com permissões intermédias. Gere os membros da sua equipa, cria convites para novos utilizadores, visualiza relatórios de actividade e acede a funcionalidades avançadas de pesquisa e organização documental.

Cliente Utilizador final com permissões básicas. Realiza *upload*, *download* e pesquisa dos seus próprios documentos. Interage com o assistente conversacional (RAG *chatbot*) para consultar informação nos documentos da organização.

2.5 Proposta de Valor

A plataforma diferencia-se das soluções concorrentes através de um conjunto único de atributos:

- **IA integrada:** processamento automático de documentos com extração de texto, geração de resumos, identificação de palavras-chave e assistente conversacional RAG;
- **Isolamento total de dados:** arquitectura multi-tenant com garantia de que nenhuma organização acede aos dados de outra, cumprindo os requisitos do RGPD;
- **IA acessível:** funcionalidades avançadas de extração e pesquisa por linguagem natural, sem necessidade de investimento em infraestrutura de IA própria;

- **Modelo SaaS escalável:** pagamento por subscrição com custos proporcionais ao uso, sem despesas de implementação ou manutenção;
- **Acesso multiplataforma:** disponível via *browser* e aplicação móvel nativa (Android).

2.6 Canais

A plataforma chega aos seus clientes através dos seguintes canais:

- **Aplicação web:** plataforma acessível via *browser*, sem necessidade de instalação, disponível em qualquer dispositivo com ligação à internet;
- **Aplicação móvel:** aplicação nativa para Android que permite acesso às funcionalidades em mobilidade;
- **API REST:** interface de programação que permite integração com sistemas de terceiros (ERP, CRM, sistemas de gestão específicos do sector);
- **Vendas directas e parcerias:** abordagem *B2B* directa a organizações dos sectores alvo, complementada por parcerias com integradores e consultoras sectoriais.

2.7 Relacionamento com Clientes

O modelo de relacionamento com os clientes da plataforma assenta em:

- **Self-service assistido:** *onboarding* guiado com documentação integrada, tutoriais e *FAQ* acessíveis diretamente na plataforma;
- **Registo por convite:** modelo controlado em que novos utilizadores só acedem após convite de um Administrador ou Gestor da sua organização, garantindo qualidade e segurança de acesso;
- **Suporte técnico:** canal de suporte via formulário e email, com resposta em tempo útil para resolução de incidentes e esclarecimento de dúvidas;
- **Atualizações contínuas:** *releases* regulares com novas funcionalidades e melhorias, comunicadas aos clientes através de *changelogs* e notificações na plataforma;
- **Comunidade e feedback:** recolha activa de *feedback* dos utilizadores para orientar o *roadmap* do produto.

2.8 Fontes de Receita

O modelo de receita baseia-se em subscrição recorrente com planos diferenciados:

- **Plano Free Trial:** acesso experimental à plataforma com limite de 3 mensagens no *chatbot* RAG; destinado a organizações que pretendam avaliar a solução antes de subscrever um plano pago;
- **Plano Starter:** acesso ao núcleo funcional da plataforma (gestão documental, pesquisa e *chatbot* RAG com limite de 10 mensagens); destinado a organizações de pequena dimensão;
- **Plano Base:** acesso ao núcleo funcional da plataforma (gestão documental, pesquisa, até 5 utilizadores e 10 GB de armazenamento); destinado a organizações de pequena dimensão;
- **Plano Profissional:** inclui processamento automático por IA, *chatbot* RAG, até 25 utilizadores e 100 GB de armazenamento; vocacionado para equipas médias;
- **Plano Empresarial:** plano sem limites fixos, com SLA garantido, suporte prioritário, integrações API avançadas e armazenamento configurável; para organizações de maior dimensão;
- **Receitas adicionais:** armazenamento extra (*add-on*), processamento de volumes elevados de documentos acima dos limites de plano, e serviços de integração personalizados.

2.9 Recursos-Chave

Os recursos fundamentais para o funcionamento e crescimento da plataforma são:

- **Plataforma tecnológica:** código-fonte da aplicação, *pipelines* CI/CD, configurações de infraestrutura (Terraform), modelos de IA configurados e *pipeline* RAG;
- **Infraestrutura cloud:** *cluster* Kubernetes na DigitalOcean com escalamento automático, bases de dados geridas PostgreSQL, *object storage* e *message broker* (RabbitMQ);
- **Equipa de desenvolvimento:** engenheiros de *software* com competências em desenvolvimento *full-stack*, DevOps, inteligência artificial e segurança;
- **Modelos de IA:** integração com LLMs (Llama 3.1), *embeddings* vectoriais e *pipeline* RAG via LangChain e ChromaDB;
- **Modelos de linguagem e *pipeline* RAG:** configuração e manutenção dos modelos de IA (Llama 3.1, *embeddings* multilingues) e da *pipeline* de processamento documental.

2.10 Atividades-Chave

As atividades centrais que sustentam o modelo de negócio são:

- **Desenvolvimento e manutenção da plataforma:** ciclos contínuos de desenvolvimento (*sprints*), testes automatizados, *deploys* sem interrupção de serviço e gestão de incidentes;
- **Operações de infraestrutura (DevOps):** monitorização contínua, gestão de *clusters* Kubernetes, optimização de custos de *cloud* e gestão de *backups*;
- **Optimização contínua dos modelos de IA:** ajuste e avaliação dos modelos de extração, resumo e geração de texto para melhorar a qualidade do processamento documental;
- **Onboarding de clientes:** configuração inicial de cada *tenant*, migração de documentos existentes e formação dos utilizadores-chave;
- **Suporte e *customer success*:** acompanhamento proactivo dos clientes para garantir adopção e renovação das subscrições.

2.11 Parcerias-Chave

O ecossistema de parcerias estratégicas inclui:

- **Fornecedores de infraestrutura *cloud*:** DigitalOcean (infraestrutura principal), com possibilidade de expansão para AWS ou Azure para redundância geográfica;
- **Fornecedores de modelos de IA:** acesso a LLMs de código aberto (Llama 3.1 via Ollama) e potencialmente APIs de fornecedores como OpenAI ou Anthropic para funcionalidades avançadas;
- **Serviços de email transaccional:** integração com Mailpit (desenvolvimento) e serviços SMTP em produção para envio de notificações e convites;
- **Integradores e consultoras:** parceiros de negócio que promovem e implementam a plataforma junto dos seus clientes;
- **Organismos reguladores:** acompanhamento das entidades reguladoras para garantir conformidade contínua com o RGPD e demais requisitos de protecção de dados.

2.12 Estrutura de Custos

Os principais custos operacionais organizam-se em:

- **Infraestrutura *cloud*:** custos fixos e variáveis com o *cluster* Kubernetes, bases de dados geridas, *object storage*, *load balancers* e largura de banda; escaláveis com o volume de utilização;
- **Custos de IA:** processamento computacional para inferência dos modelos de linguagem e geração de *embeddings*; custo por documento processado e por consulta RAG;

- **Equipa de desenvolvimento e operações:** salários da equipa técnica (desenvolvimento, DevOps, *data science*), principal componente de custo fixo;
- **Serviços de terceiros:** licenças de *software*, ferramentas de monitorização, serviços de email transaccional e processadores de pagamento;
- **Vendas e *marketing*:** aquisição de clientes, participação em eventos sectoriais e produção de conteúdo de posicionamento;
- **Suporte e *customer success*:** equipa de suporte técnico e acompanhamento de clientes, custo que cresce proporcionalmente com a base de clientes.

3. Requisitos do Cliente

Os requisitos da plataforma encontram-se organizados em três categorias: funcionais (o que o sistema faz), não funcionais (qualidade, segurança e manutenção) e requisitos de *hardware* e *software*.

3.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades concretas que o sistema deve disponibilizar aos seus utilizadores.

3.1.1 RF1:Gestão de Utilizadores e Autenticação

RF1.1:Autenticação de Utilizadores

- *Login* através de email e *password*;
- Autenticação baseada em tokens JWT (*JSON Web Tokens*) para manutenção de sessões *stateless*;
- Validação de credenciais contra base de dados PostgreSQL;
- *Logout* com invalidação imediata do token (*token blacklisting*);
- *Refresh tokens* para renovação automática de sessões sem re-autenticação;
- Durações dos tokens de acesso e *refresh* configuráveis via variáveis de ambiente (`ACCESS_TOKEN_EXPIRY` e `REFRESH_TOKEN_EXPIRY`), com valores por defeito de 7 e 30 dias, respetivamente;
- Registo de todas as tentativas de autenticação (sucessos e falhas) para auditoria.

RF1.2:Gestão de Perfis de Acesso (RBAC)

A plataforma implementa três níveis hierárquicos de perfil com permissões diferenciadas:

- **Cliente:** acesso limitado aos seus próprios documentos, com funcionalidades básicas de *upload*, *download* e pesquisa; sem permissões de gestão de utilizadores;

- **Gestor:** herda todas as permissões de Cliente, acrescidas de gestão de utilizadores da organização, criação de convites e visualização de relatórios de actividade;
- **Administrador:** acesso total, incluindo configurações globais da organização, gestão completa de utilizadores, acesso a *logs* de auditoria e configuração de integrações externas.

RF1.3:Registo por Convite

- Apenas utilizadores com perfil Gestor ou Administrador podem criar convites;
- Cada convite gera um código único com validade de 48 horas;
- Novos utilizadores completam o registo através do *link* de convite recebido por email;
- O convite é marcado como utilizado após registo bem-sucedido, não podendo ser reutilizado.

3.1.2 RF2:Gestão Documental

RF2.1:Upload de Documentos

- Aceitação de múltiplos formatos: PDF, DOCX, TXT, XLSX, PNG, JPG;
- Validação do tipo MIME real do ficheiro, não apenas da extensão;
- Imposição de tamanho máximo por ficheiro e suporte para *upload* em lote;
- Geração automática de metadados: nome, tamanho, tipo MIME, data de *upload*, UUID do utilizador;
- Armazenamento de ficheiros em *object storage* (MinIO / DigitalOcean Spaces / AWS S3).

RF2.2:Processamento Automático por IA

- Extração automática de texto após *upload*;
- Geração de resumo automático e extração de palavras-chave;
- Criação de *embeddings* vectoriais para pesquisa semântica e *chatbot* RAG.

RF2.3:Pesquisa e Acesso

- Pesquisa por texto completo nos metadados e conteúdo dos documentos;
- Filtragem por departamento, data, tipo de documento e utilizador;
- *Download* e partilha de documentos entre utilizadores da mesma organização.

RF2.4:Edição de Documentos

- Edição dos metadados de documentos existentes, nomeadamente *nome original*, *descrição* e *departamento* associado;
- Suporte para remoção da associação a um departamento (valor `null`).

3.1.3 RF3:Assistente Conversacional (RAG Chatbot)

- Interface de linguagem natural para interrogar documentos da organização, acessível via `POST /api/documents/{documentId}/chat`;
- Respostas geradas com base em recuperação semântica (*Retrieval-Augmented Generation*): o sistema identifica os excertos mais relevantes do documento através de similaridade de *embeddings* e envia-os como contexto ao modelo de linguagem;
- *Embeddings* gerados com o modelo HuggingFace paraphrase-multilingual-MiniLM-L12-v2, armazenados em ChromaDB por documento;
- Respostas contextualizadas exclusivamente com base nos documentos do *tenant* do utilizador autenticado;
- Isolamento total: o *chatbot* não acede a documentos de outras organizações.

RF3.1:Sistema de Quotas por Plano de Subscrição

- O acesso ao *chatbot* é controlado por um sistema de quotas associado ao plano de subscrição do utilizador;
- Plano **Free Trial**: máximo de 3 mensagens;
- Plano **Starter**: máximo de 10 mensagens;
- O sistema regista o número de mensagens consumidas por utilizador, permitindo controlo granular do uso da funcionalidade de IA;
- Quando o limite é atingido, a API retorna HTTP 403 com o código `limit_reached`, impedindo novas interações até que o plano seja atualizado.

3.1.4 RF4:Multi-tenancy e Isolamento de Dados

- Identificação do *tenant* (organização) através de subdomínio HTTP ou *header* personalizado;
- Filtragem automática por `organizationId` em todas as *queries* à base de dados;
- Prevenção de qualquer acesso cruzado entre organizações distintas.

3.1.5 RF5: Integrações Externas

- Envio de emails de convite e notificações via SMTP;
- *Templates* de email configuráveis por organização;
- Registo de histórico de emails enviados.

3.1.6 RF6: Gestão de Departamentos

- Os departamentos estão associados a uma organização e permitem categorizar utilizadores e documentos;
- As operações disponíveis sobre departamentos são as seguintes:

Todos os *endpoints* têm como prefixo `/api/organizations/{orgId}`.

Método	Sufixo	Descrição	Permissão
GET	<code>/departments</code>	Listar departamentos	Autenticado
GET	<code>/departments/{id}</code>	Obter departamento	Autenticado
POST	<code>/departments</code>	Criar departamento	Administrador
PATCH	<code>/departments/{id}</code>	Atualizar departamento	Administrador
DELETE	<code>/departments/{id}</code>	Eliminar departamento	Administrador

Tabela 3.1: Endpoints do módulo de gestão de departamentos

3.2 Requisitos Não Funcionais

Os requisitos não funcionais definem as características de qualidade, segurança, manutenção e operação da plataforma.

3.2.1 RNF1: Desempenho

- Tempos de resposta rápidos para operações síncronas da API (pedidos HTTP);
- Operações de longa duração (envio de emails, processamento de documentos) executadas de forma assíncrona, sem impacto na responsividade da interface;
- Suporte para múltiplos utilizadores simultâneos sem degradação perceptível do serviço.

3.2.2 RNF2: Escalabilidade

- Escalamento horizontal via Kubernetes *Horizontal Pod Autoscaler* (HPA);
- Aumento automático de réplicas por microserviço baseado em métricas de CPU (> 70%) e memória (> 80%);

- Escalamento automático de 1 a 3 réplicas em configuração padrão;
- Capacidade de escalar para 1.000+ utilizadores simultâneos com ajustes de infraestrutura.

3.2.3 RNF3:Segurança

- Armazenamento de *passwords* com *hash* bcrypt (fator de custo = 10);
- Comunicação externa via HTTPS/TLS 1.3 ou superior;
- Tokens JWT com expiração configurável via variáveis de ambiente, com valor por defeito de 7 dias para tokens de acesso e 30 dias para *refresh tokens*;
- *Rate limiting*: máximo de 5 tentativas de autenticação por minuto por IP;
- Proteção contra SQL Injection, XSS e CSRF através de validação e sanitização de *inputs*;
- *Headers* de segurança HTTP implementados: CSP, HSTS, X-Frame-Options;
- Segredos (credenciais, JWT *secrets*, API *keys*) armazenados em Kubernetes Secrets;
- Registo de todas as operações sensíveis em *logs* de auditoria.

3.2.4 RNF4:Disponibilidade

- Disponibilidade mínima de 99% (SLA);
- *Health checks* para deteção automática de falhas (endpoint */health*);
- *Deployments* com *rolling updates*, garantindo zero *downtime*;
- *Rollback* automático em caso de falha de *health checks* na nova versão;
- Redundância mínima de 2 réplicas por microserviço em produção.

3.2.5 RNF5:Manutenibilidade

- Adesão a convenções e boas práticas de código (ESLint, princípios *clean code*);
- Cobertura de testes unitários e de integração superior a 80%;
- Documentação de todas as APIs públicas segundo o standard OpenAPI 3.0;
- *Logs* estruturados em formato JSON para facilitar análise e correlação;
- Versionamento semântico automático para *releases* (Semantic Release);
- Infraestrutura como Código (Terraform) para reprodutibilidade total dos ambientes.

3.2.6 RNF6:Observabilidade

- Exposição de métricas no formato Prometheus (endpoint `/metrics`);
- *Dashboards* de monitorização em tempo real via Grafana;
- Registo estruturado de eventos relevantes para correlação de incidentes;
- *Alertas* automáticos para condições de erro ou degradação de desempenho.

3.2.7 RNF7:Cópias de Segurança (*Backups*)

- *Backups* automáticos diários da base de dados PostgreSQL, retidos durante 7 dias;
- Possibilidade de restauro pontual (*point-in-time recovery*) em caso de perda ou corrupção de dados;
- Replicação dos ficheiros armazenados no *object storage* para garantir durabilidade;
- Procedimentos documentados de *disaster recovery* e janelas de manutenção configuráveis.

3.3 Requisitos de *Hardware* e *Software*

3.3.1 Requisitos de *Hardware* (Servidor / Infraestrutura)

A plataforma opera em infraestrutura *cloud* gerida, com os seguintes requisitos mínimos para o ambiente de produção:

Componente	Especificação Mínima
<i>Cluster</i> Kubernetes	1–3 nós com 2 vCPU e 2 GB RAM cada (DigitalOcean s-2vcpu-2gb)
Base de Dados	PostgreSQL 16 gerida, 1 vCPU, 1 GB RAM, 10 GB SSD
<i>Object Storage</i>	Compatível com API S3; capacidade inicial de 100 GB
<i>Message Broker</i>	RabbitMQ; 1 vCPU, 512 MB RAM
<i>Load Balancer</i>	Gerido pelo fornecedor <i>cloud</i> ; IP público estático
Rede	Largura de banda mínima de 100 Mbps; latência interna <5 ms entre nós

Tabela 3.2: Requisitos de *hardware* da infraestrutura de produção

Para o processamento de IA (modelos LLM locais via Ollama), é recomendado:

Componente	Especificação Recomendada
CPU	8+ cores para inferência em CPU; GPU NVIDIA (CUDA) para inferência acelerada
RAM	Mínimo 16 GB; recomendado 32 GB para modelos Llama 3.1 7B
Armazenamento	SSD NVMe com pelo menos 50 GB para modelos e índices vectoriais

Tabela 3.3: Requisitos de *hardware* para o serviço de IA (AI Service)

3.3.2 Requisitos de *Hardware* (Cliente / Utilizador Final)

Para acesso via *browser* (aplicação web):

- Qualquer dispositivo com ligação à internet e *browser* moderno;
- Largura de banda mínima recomendada de 5 Mbps para *upload* de documentos.

Para acesso via aplicação móvel:

- **Android:** dispositivo com Android 10 (API level 29) ou superior, com pelo menos 2 GB RAM.

3.3.3 Requisitos de *Software* (Servidor)

Componente	Versão	Função
Node.js	18 LTS	<i>Runtime</i> dos microsserviços IDP, Organization e Notification
NestJS / Express.js	10.x / 4.18	<i>Frameworks</i> web dos microsserviços
Python	3.11+	<i>Runtime</i> do AI Service
FastAPI	0.100+	<i>Framework</i> web do AI Service
PostgreSQL	16	Base de dados relacional (uma por microsserviço)
RabbitMQ	3.12+	<i>Message broker</i> para comunicação assíncrona
MinIO	AGPL-3.0	<i>Object storage</i> compatível com API S3
Kubernetes	1.28+	Orquestração de contentores
Docker	24+	Contentorização dos microsserviços
Nginx	1.24+	<i>API Gateway</i> e <i>reverse proxy</i>
Prometheus	2.47+	Recolha de métricas
Grafana	10+	Visualização de métricas e <i>dashboards</i>
Terraform	1.6+	Infraestrutura como Código
LangChain	0.1+	<i>Pipeline</i> RAG
ChromaDB	0.4+	Base de dados vectorial para <i>embeddings</i>
Ollama	0.1+	Servidor local para LLMs (Llama 3.1)
sentence-transformers	–	Geração de <i>embeddings</i> multilingues (modelo paraphrase-multilingual-MiniLM-L12-v2)

Tabela 3.4: Requisitos de *software* do lado do servidor

3.3.4 Requisitos de *Software* (Cliente)

Componente	Requisito
<i>Browser</i> (aplicação web)	Chrome 110+, Firefox 115+, Safari 16+, Edge 110+
Sistema operativo móvel (Android)	Android 10 (API 29) ou superior
Ligação à internet	Necessária para todas as funcionalidades

Tabela 3.5: Requisitos de *software* do lado do cliente

4. Wireframes da Aplicação Móvel

Os *wireframes* da aplicação móvel foram desenvolvidos na ferramenta Figma e constituem representações esquemáticas das interfaces, focando-se exclusivamente na estrutura, disposição dos elementos (*layout*) e fluxos de navegação, sem detalhes visuais como cores ou tipografia final.

O objectivo é validar a hierarquia de informação e a usabilidade das interfaces antes da implementação visual, garantindo que o utilizador consegue navegar e interagir com as funcionalidades de forma intuitiva.

O projeto completo pode ser consultado na plataforma Figma:

<https://www.figma.com/files/team/1562161671157189849/project/478430828/ScriptumAI>

4.1 Fluxo de Autenticação

O ponto de entrada na aplicação é o fluxo de autenticação, essencial para garantir o acesso seguro e a segregação de dados por *tenant*. Como ilustrado na Figura 4.1, este fluxo compreende os seguintes ecrãs:

- **Ecrã Inicial:** apresenta o logótipo da aplicação e as opções primárias de *Login* (Entrar) e *Sign Up* (Registar);
- **Login:** formulário simples para inserção de credenciais (email e *password*);
- **Registo:** formulário para criação de nova conta, com confirmação de *password*;
- **Reset Password:** ecrã acessível a partir do *Login* para recuperação de *password*.

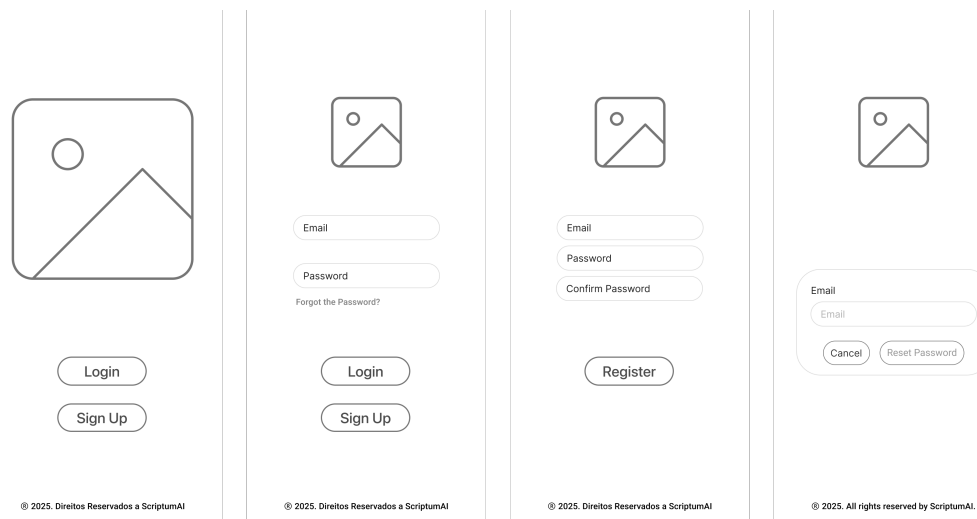


Figura 4.1: Wireframes do fluxo de Autenticação (Ecrã inicial, Login, Registo e Recuperação de *Password*).

4.2 Gestão de Documentos

O fluxo principal da aplicação é a gestão de documentos, detalhada na Figura 4.2.

O ecrã **Documents** é o *dashboard* principal. Permite visualizar rapidamente os documentos recentes e filtrá-los por departamento.

O fluxo de **New Documents** foi desenhado para ser intuitivo, com uma área de *upload* e campos para metadados essenciais (título e descrição).

O ecrã **Document Details** exibe a informação extraída e processada pela IA (Resumo, Palavras-chave) e disponibiliza acções como *Download* e *Share*.

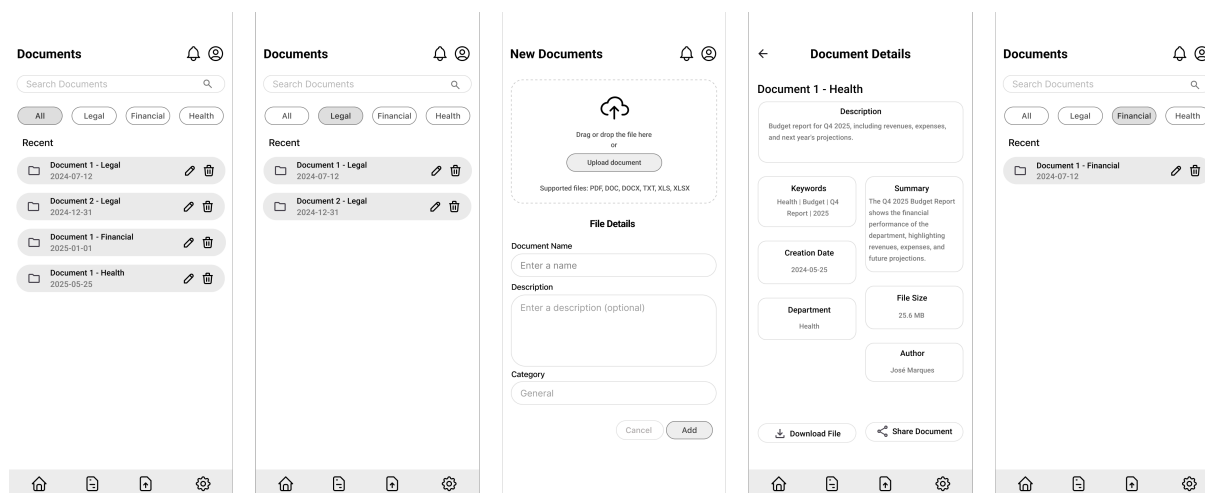


Figura 4.2: Wireframes do fluxo de Gestão de Documentos (Listagem, Filtros, Adição e Detalhe).

4.3 Definições e Gestão de Perfil

A Figura 4.3 apresenta os ecrãs de configuração pessoal e gestão de conta.

O ecrã **Settings** serve como ecrã central de configuração, dando acesso a:

- **Edit Profile**: alteração de dados pessoais;
- **Theme**: selecção entre modo Claro, Escuro ou Sistema;
- **Notifications**: activação ou desactivação de alertas;
- **Language**: alternância entre idiomas suportados (EN/PT);
- **Help**: acesso a recursos de suporte e formulário de contacto.

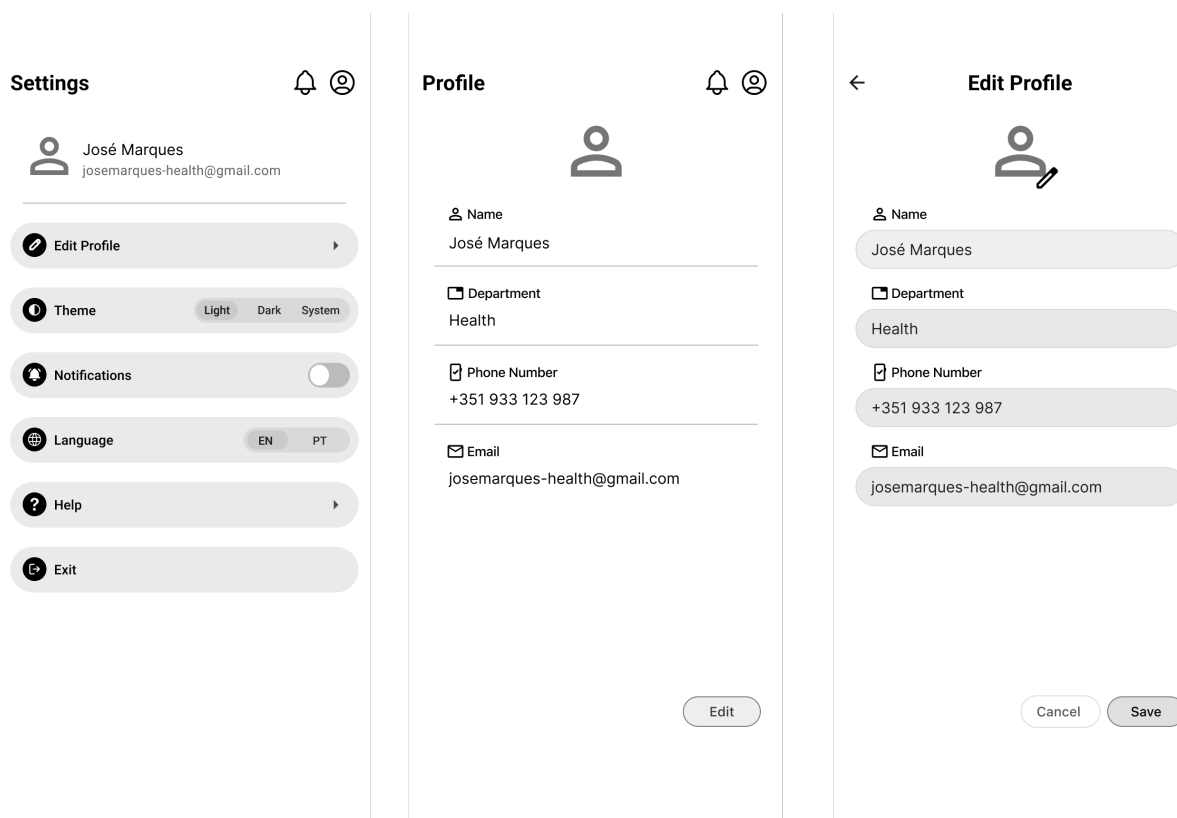


Figura 4.3: Wireframes do fluxo de Definições e Gestão de Perfil.

4.4 Gestão de Utilizadores (Administração)

Para utilizadores com permissões de administração, foi desenhado um fluxo de gestão de equipa (Figura 4.4). A secção **Manage Users** divide-se em dois separadores:

- **Users:** lista os membros actuais da organização, permitindo editar dados e permissões ou remover utilizadores;
- **Invites:** lista os convites pendentes com a respectiva data de expiração, permitindo controlo rigoroso sobre quem acede ao sistema.

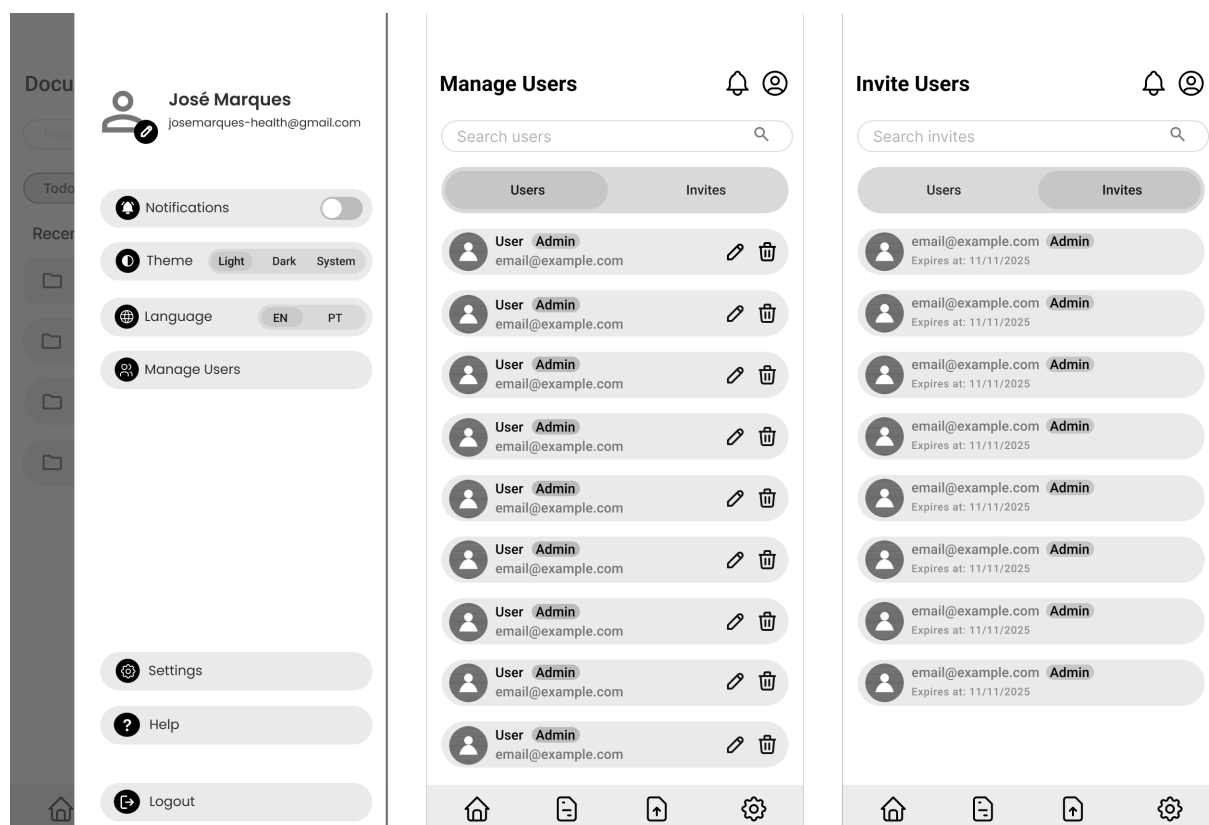


Figura 4.4: Wireframes do fluxo de Gestão de Utilizadores (*Sidebar*, Lista de Utilizadores e Convites).

4.5 Notificações

O centro de **Notifications** (Figura 4.5) fornece *feedback* assíncrono ao utilizador sobre eventos importantes, como a conclusão do processamento de um documento ou a partilha de ficheiros.

A interface permite filtrar entre notificações **Unread** (Não Lidas) e **All Notifications** (Todas). Foi contemplado o estado vazio, que informa o utilizador de forma clara quando não existem notificações por ler.

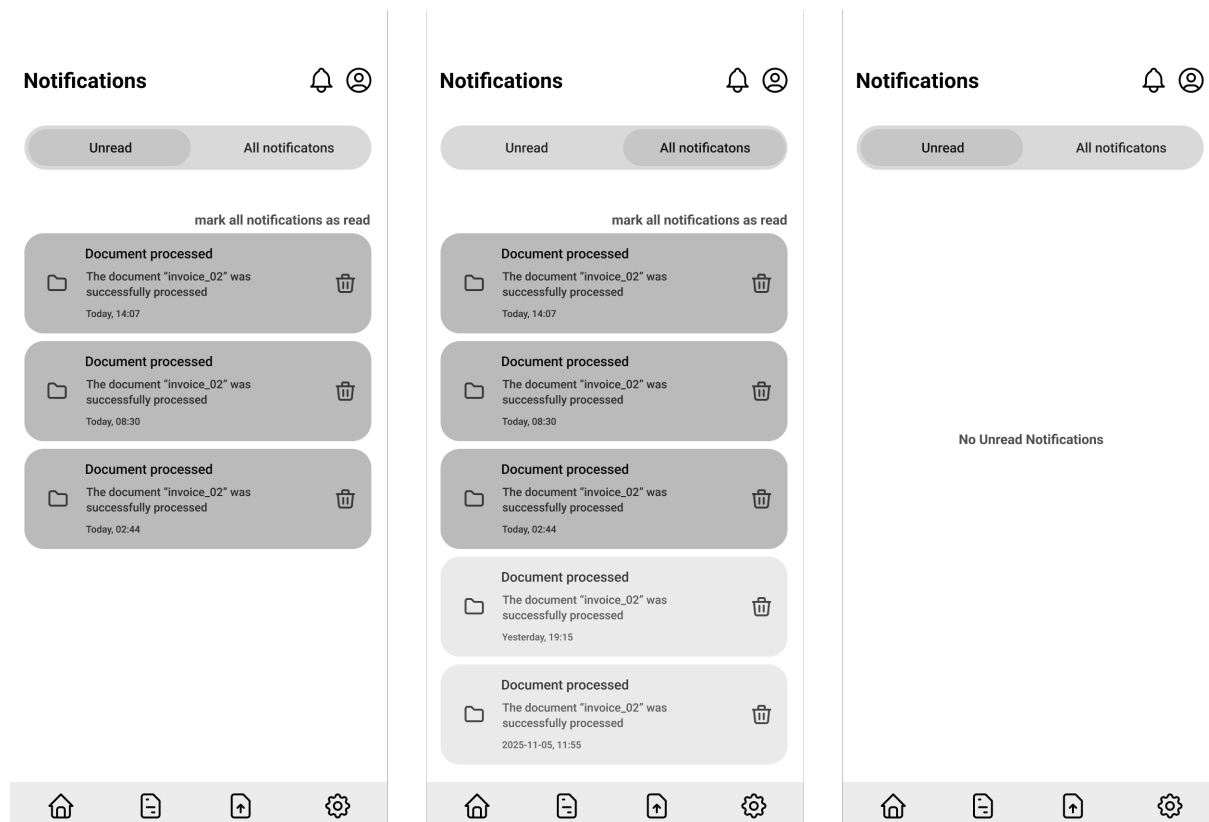


Figura 4.5: Wireframes do Centro de Notificações (Não Lidas, Todas e Estado Vazio).

4.6 Mockups de Alta Fidelidade

Após a validação dos *wireframes*, foi desenvolvido o protótipo de alta fidelidade (*hi-fi*) com a identidade visual completa da aplicação. As seguintes figuras ilustram os principais ecrãs já com cores, tipografia e componentes finais.

4.6.1 Fluxo de Autenticação

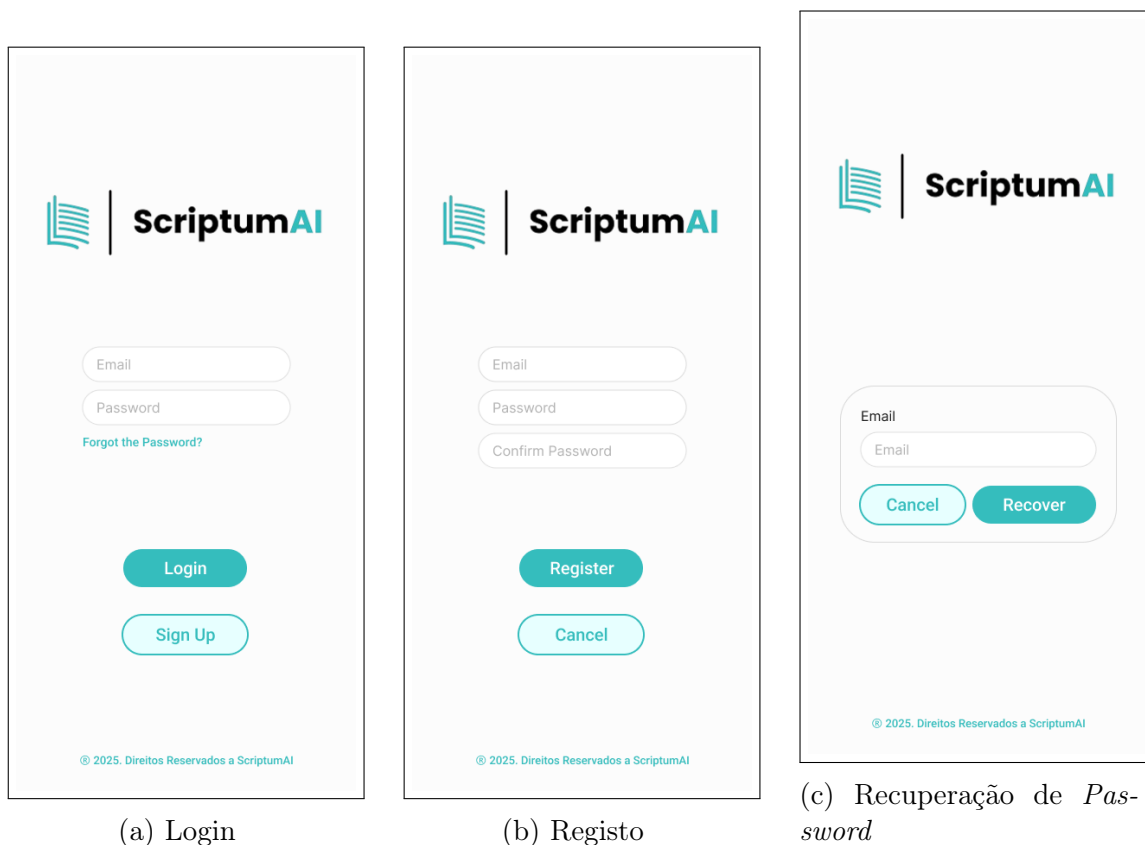


Figura 4.6: Mockups do fluxo de autenticação.

4.6.2 Ecrã Principal e Gestão de Documentos

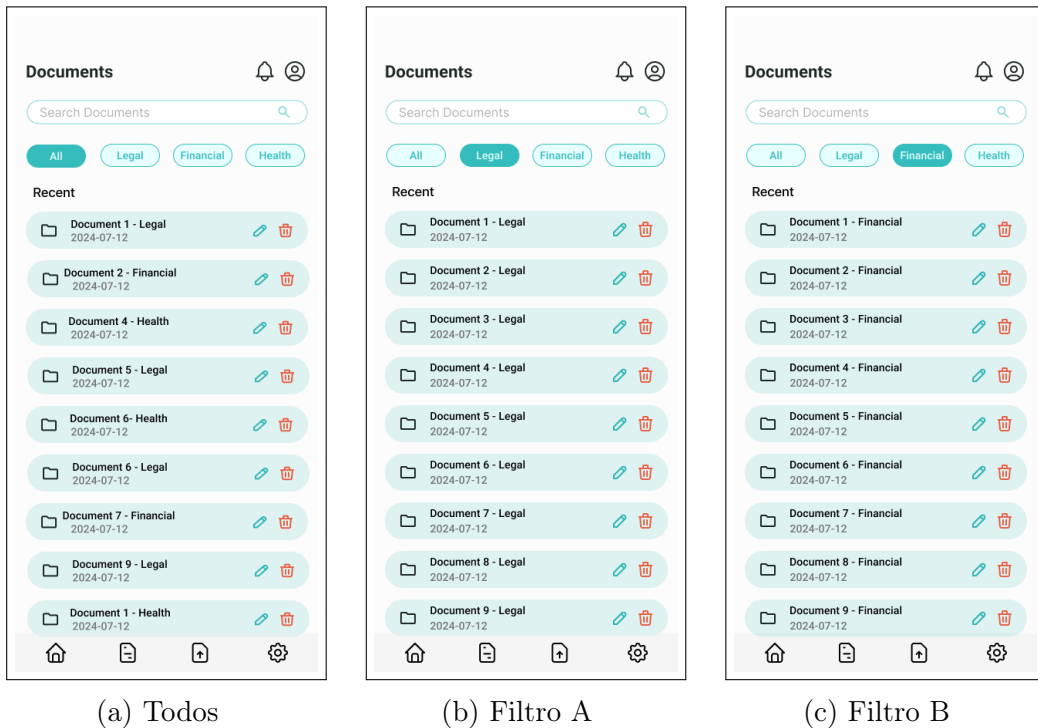


Figura 4.7: Mockups do ecrã principal com filtros por departamento.

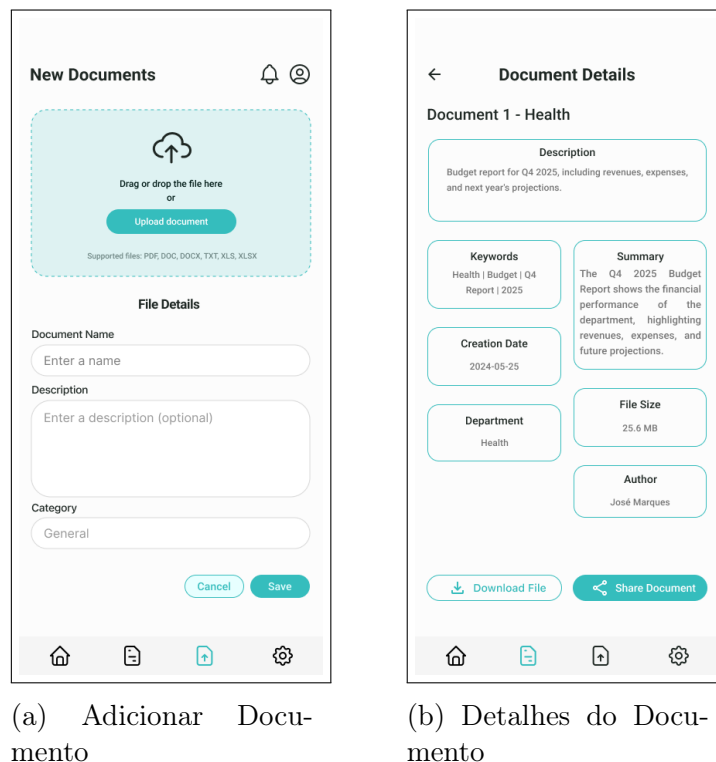
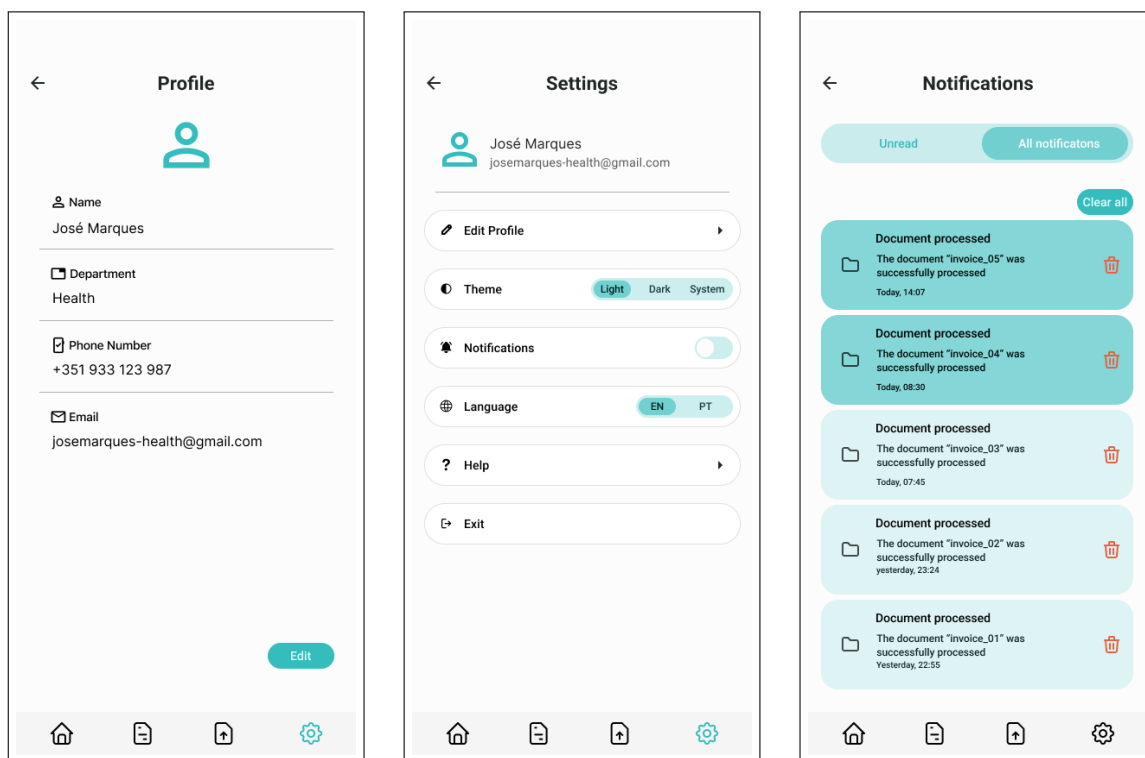


Figura 4.8: Mockups da gestão de documentos.

4.6.3 Perfil, Definições e Notificações



(a) Perfil

(b) Definições

(c) Notificações

Figura 4.9: Mockups de perfil, definições e notificações.

4.6.4 Gestão de Utilizadores (Administração)

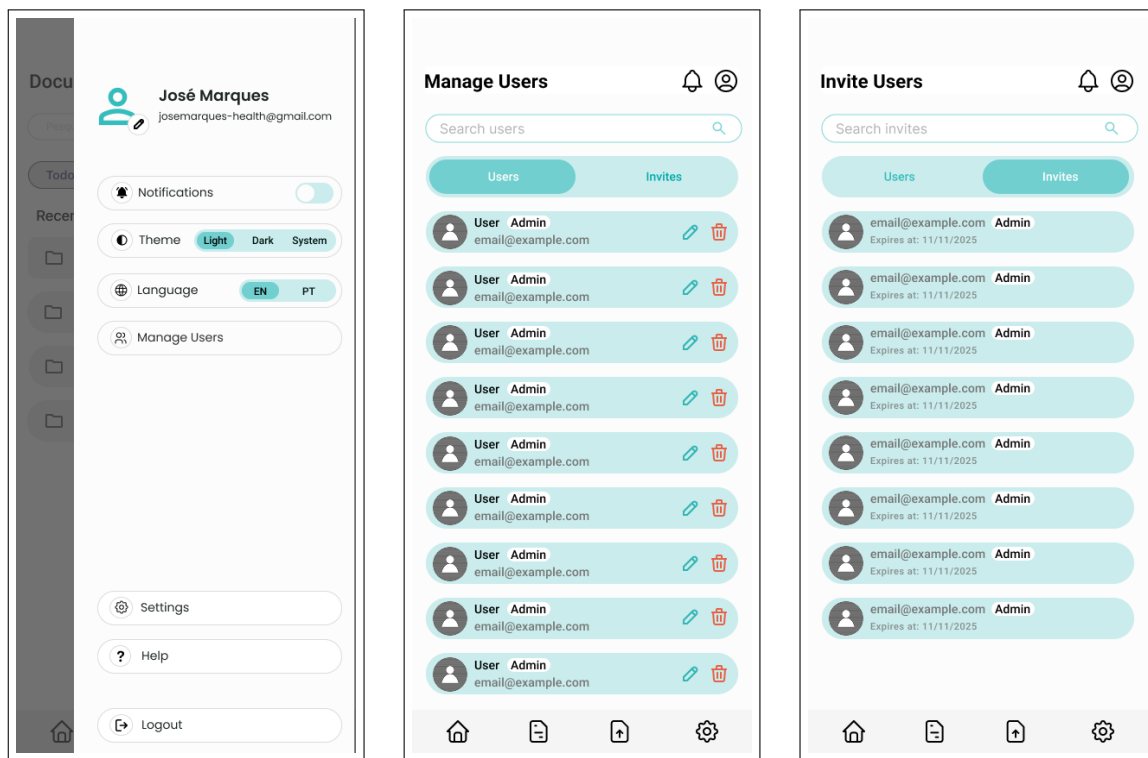
(a) *Sidebar Admin*(b) *Utilizadores*(c) *Convites*

Figura 4.10: Mockups da gestão de utilizadores (Administração).

5. Especificação

A especificação técnica da plataforma abrange o modelo de domínio, o diagrama de classes, os diagramas de casos de uso com respectiva descrição e os diagramas de sequência dos fluxos principais.

5.1 Modelo de Domínio

O modelo de domínio do **ScriptumAI** é composto pelas entidades centrais do negócio e pelas relações que as interligam. Dado que a plataforma segue o padrão *Database per Service*, cada microsserviço é dono das suas próprias entidades; as relações entre serviços são mantidas através de referências lógicas por UUID, sem *foreign keys* físicas entre bases de dados distintas.

As entidades principais são as seguintes:

5.1.1 Entidade: Organization

Representa um *tenant* (organização/empresa) na plataforma.

Atributo	Descrição
id	UUID — Identificador único (PK)
name	String(200) — Nome da organização
subdomain	String(50) — Subdomínio único
plan	Enum — FREE_TRIAL, STARTER, BASE, PROFISSIONAL, EMPRESARIAL
logoUrl	String(500) — URL do logótipo
settings	JSON — Configurações personalizadas
isActive	Boolean — Organização ativa/inativa
createdAt	Timestamp — Data de criação
updatedAt	Timestamp — Última atualização

5.1.2 Entidade: User

Representa um utilizador registado na plataforma.

Atributo	Descrição
id	UUID — Identificador único (PK)
organizationId	UUID — Referência lógica para Organization
email	String(255) — Email único
passwordHash	String(255) — Hash bcrypt
role	Enum — CLIENTE, GESTOR, ADMIN
firstName	String(100) — Primeiro nome
lastName	String(100) — Apelido
chatQuotaUsed	Integer — Mensagens RAG consumidas
isActive	Boolean — Conta ativa/inativa
lastLoginAt	Timestamp — Último login
createdAt	Timestamp — Data de criação

5.1.3 Entidade: Document

Representa um documento carregado e processado pela plataforma.

Atributo	Descrição
id	UUID — Identificador único (PK)
organizationId	UUID — Referência lógica para Organization
userId	UUID — Referência lógica para User (autor do upload)
departmentId	UUID — Referência lógica para Department (opcional)
originalName	String(500) — Nome original do ficheiro
description	String — Descrição do documento
fileSize	BigInteger — Tamanho em bytes
mimeType	String(100) — Tipo MIME
storageKey	String(1000) — Chave no object storage
extractedText	Text — Texto extraído por IA
summary	Text — Resumo gerado por IA
keywords	Array<String> — Palavras-chave extraídas
aiProcessed	Boolean — Processamento por IA concluído
uploadedAt	Timestamp — Data de upload

5.1.4 Entidade: Invite

Armazena convites pendentes para incorporação de novos utilizadores.

Atributo	Descrição
id	UUID — Identificador único (PK)
organizationId	UUID — Referência lógica para Organization
createdBy	UUID — Referência lógica para User criador
email	String(255) — Email do convidado
role	Enum — Role a atribuir ao convidado
code	UUID — Código único do convite
expiresAt	Timestamp — Data de expiração (48 horas)
usedAt	Timestamp — Data de utilização (NULL se não usado)
createdAt	Timestamp — Data de criação

5.1.5 Entidade: Department

Permite categorizar utilizadores e documentos dentro de uma organização.

Atributo	Descrição
id	UUID — Identificador único (PK)
organizationId	UUID — Referência lógica para Organization
name	String(200) — Nome do departamento
description	String — Descrição do departamento
createdAt	Timestamp — Data de criação

5.1.6 Entidade: TokenBlacklist

Regista tokens JWT invalidados após logout, implementando revogação imediata de sessões.

Atributo	Descrição
id	UUID — Identificador único (PK)
token	String(1000) — Token JWT completo
userId	UUID — Referência lógica para User
expiresAt	Timestamp — Expiração do token
createdAt	Timestamp — Adição à blacklist

5.1.7 Relações Lógicas entre Entidades

A tabela seguinte resume as dependências lógicas entre os microsserviços, mantidas através de UUIDs em vez de *foreign keys* físicas:

Serviço Origem	Serviço Destino	Relação Lógica
Organization Service	IDP Service	Membership.userId → User.id
Document Service	IDP Service	Document.userId → User.id
Document Service	Organization Service	Document.organizationId → Organization.id
Document Service	Organization Service	Document.departmentId → Department.id
Notification Service	IDP Service	Notification.userId → User.id
Notification Service	Organization Service	Template.organizationId → Organization.id

Tabela 5.1: Relações lógicas entre microsserviços

5.2 Diagrama de Classes

Dado que o **ScriptumAI** adota uma arquitectura de microsserviços com o padrão *Database per Service*, os diagramas entidade-relação de cada serviço correspondem, funcionalmente, aos diagramas de classes das suas entidades de domínio. As figuras seguintes apresentam esses modelos por serviço.

5.2.1 IDP Service

O IDP Service é responsável pela autenticação e autorização de utilizadores, gestão de tokens JWT e auditoria de acessos. As entidades **User**, **TokenBlacklist** e **AuthLog** formam o seu modelo de dados.

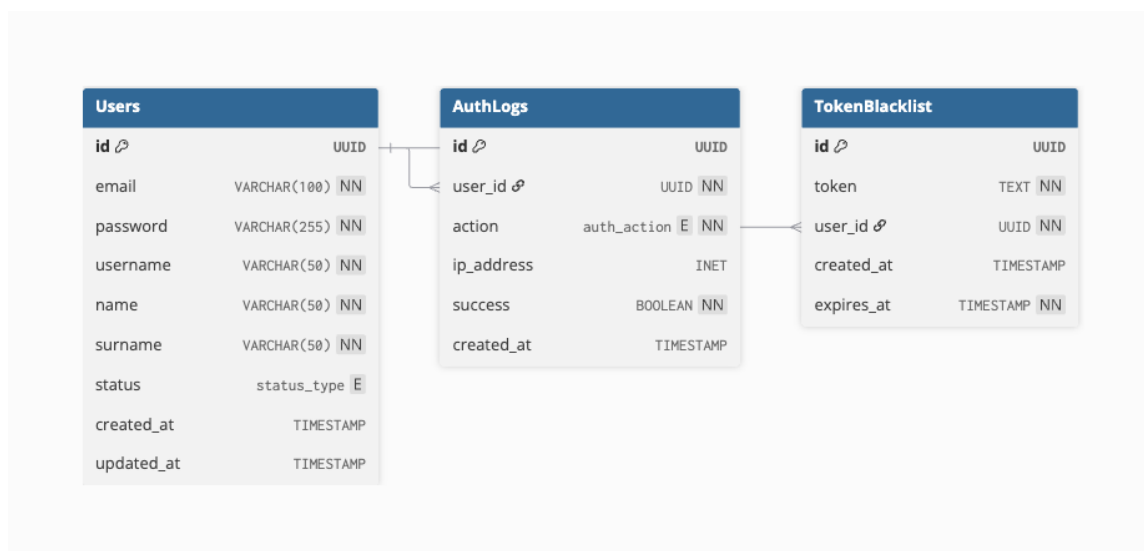


Figura 5.1: Diagrama de classes do IDP Service

5.2.2 Organization Service

O Organization Service gere organizações (*tenants*), membros e o sistema de convites. As suas entidades centrais são **Organization**, **Membership**, **Invite** e **Department**.

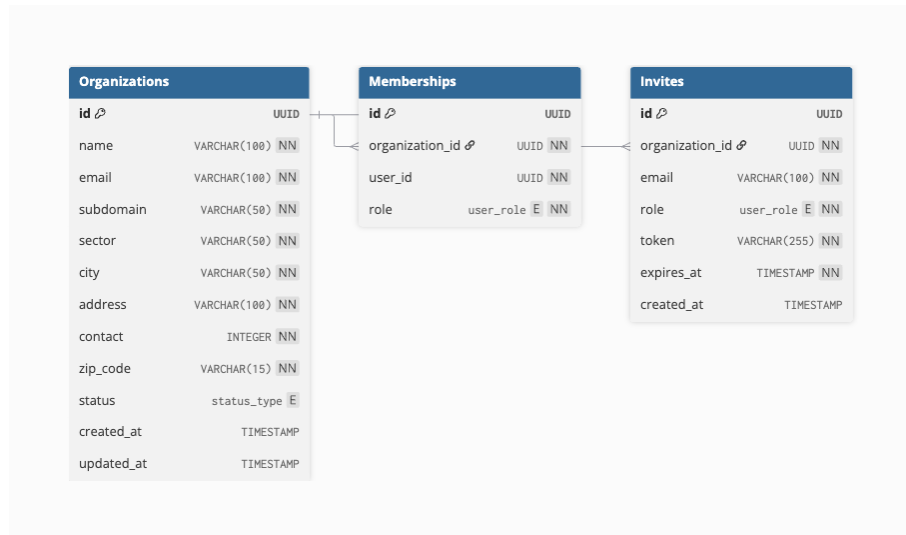


Figura 5.2: Diagrama de classes do Organization Service

5.2.3 Document Service

O Document Service gere o ciclo de vida dos documentos: armazenamento, processamento por IA e edição de metadados. A entidade central é **Document**.

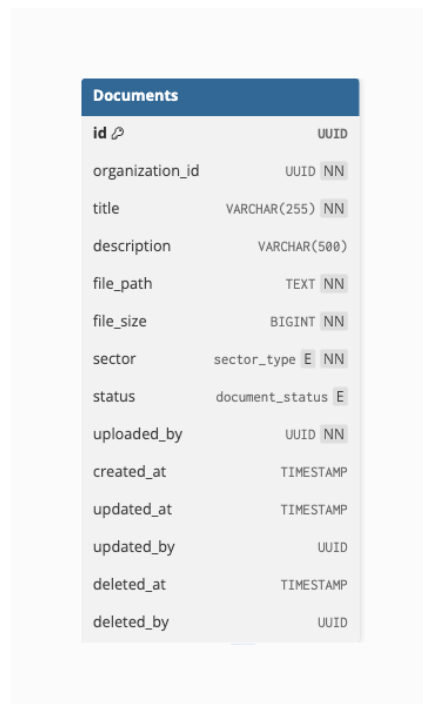


Figura 5.3: Diagrama de classes do Document Service

5.2.4 Notification Service

O Notification Service gere o envio de emails, templates configuráveis e histórico de notificações. As entidades principais são **Notification**, **NotificationTemplate** e **NotificationLog**.

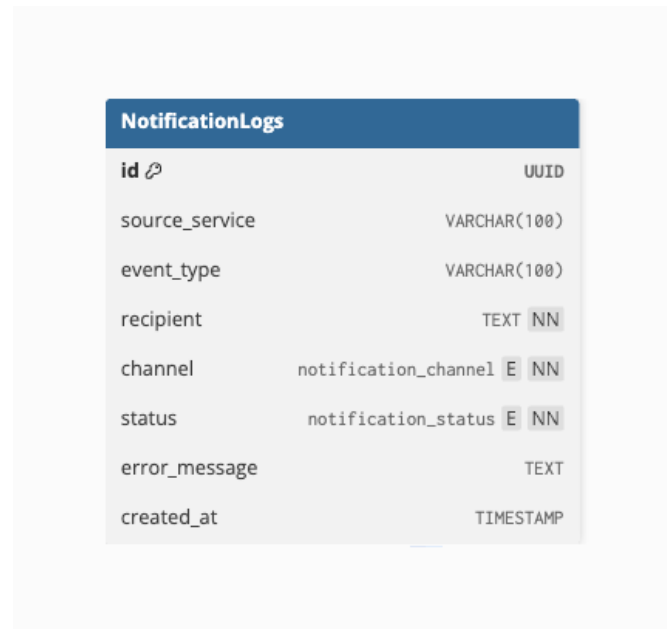


Figura 5.4: Diagrama de classes do Notification Service

5.3 Diagramas de Casos de Uso

Esta secção descreve os casos de uso mais relevantes da plataforma, identificando atores, pré-condições, fluxos e pós-condições.

5.3.1 CU1: Login de Utilizador

Ator: Cliente, Gestor ou Administrador

Pré-condições: O utilizador possui credenciais válidas registadas na plataforma.

Fluxo Principal:

1. Utilizador acede à página de login;
2. Insere email e *password* no formulário;
3. A aplicação valida o formato dos campos;
4. Envio de requisição POST `/api/auth/login`;
5. Verificação da existência do email na base de dados;
6. Comparação da *password* fornecida com o *hash* armazenado (bcrypt);

7. Geração de token JWT com `userId`, `organizationId` e `role`;
8. Registo da ação em `AuthLog`;
9. Retorno do token JWT e redirecionamento para o *dashboard*.

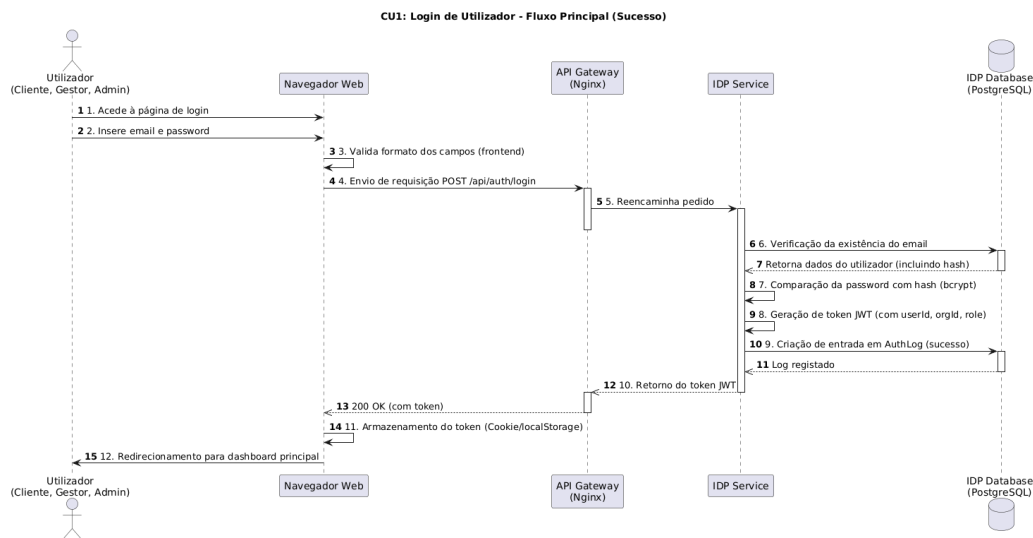


Figura 5.5: CU1 — Login: fluxo principal (sucesso)

Fluxo Alternativo 1: Credenciais Inválidas:

- 6a. Detecção de *password* incorreta;
- 6b. Registo da tentativa falhada em `AuthLog`;
- 6c. Retorno de HTTP 401: “Email ou *password* incorretos”;
- 6d. Máximo de 5 tentativas por minuto por IP (*rate limiting*).

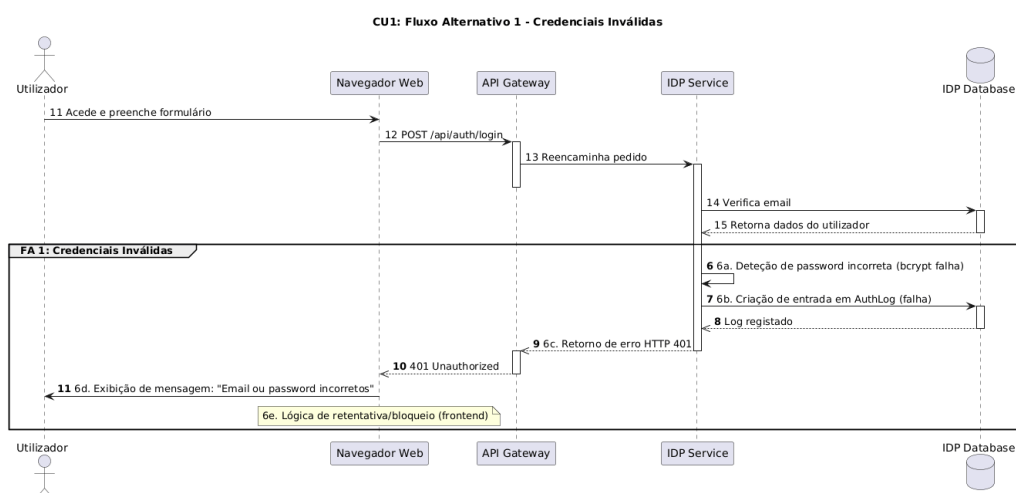


Figura 5.6: CU1 — Login: credenciais inválidas

Fluxo Alternativo 2: Conta Desactivada:

- 5a. Detecção de conta inativa (`isActive = false`);
- 5b. Retorno de HTTP 403: “Conta desativada. Contacte o administrador”.

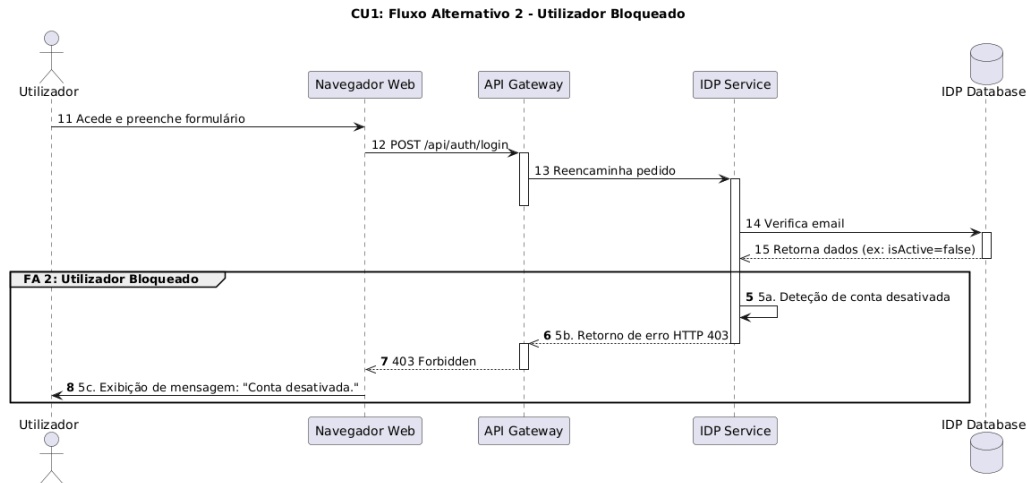


Figura 5.7: CU1 — Login: conta desativada

Pós-condições: Utilizador autenticado com acesso aos recursos protegidos pela sua *role*.

5.3.2 CU2: Upload de Documento

Ator: Cliente, Gestor ou Administrador

Pré-condições: Utilizador autenticado na plataforma.

Fluxo Principal:

1. Utilizador acede à funcionalidade de *upload*;
2. Seleciona um ou mais ficheiros do sistema local;
3. Validação do tipo MIME e tamanho do ficheiro;
4. Envio de requisição `POST /api/documents/upload` (multipart/form-data);
5. Extração do `organizationId` e `userId` do token JWT;
6. Armazenamento do ficheiro no *object storage* (MinIO);
7. Publicação do evento `document.uploaded` no RabbitMQ;
8. O AI Service processa o documento: extração de texto, geração de resumo, extração de palavras-chave e criação de *embeddings*;
9. Resposta HTTP 201 com metadados do documento.

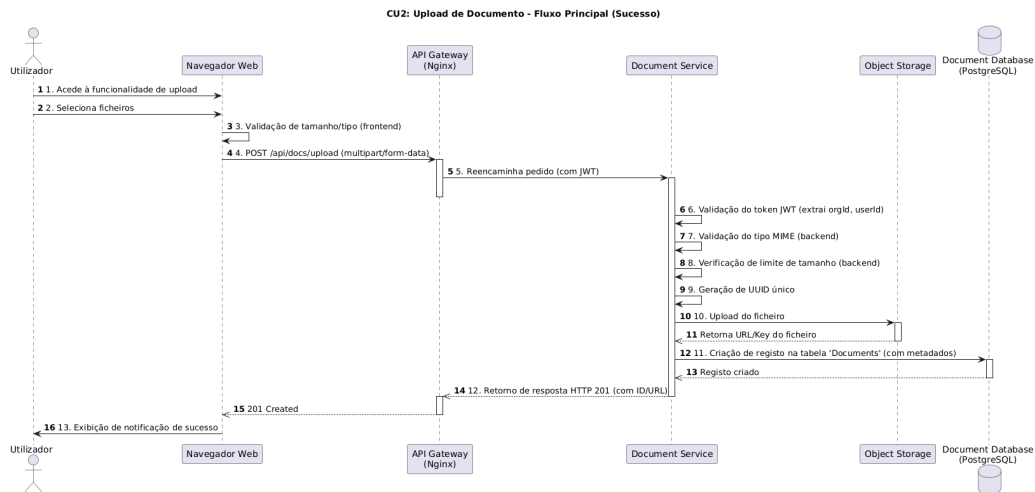


Figura 5.8: CU2 — Upload de Documento: fluxo principal (sucesso)

Fluxo Alternativo 1: Ficheiro Inválido:

- 3a. Tipo MIME não suportado;
- 3b. HTTP 400: “Tipo de ficheiro não suportado. Formatos aceites: PDF, DOCX, TXT, XLSX, PNG, JPG”.

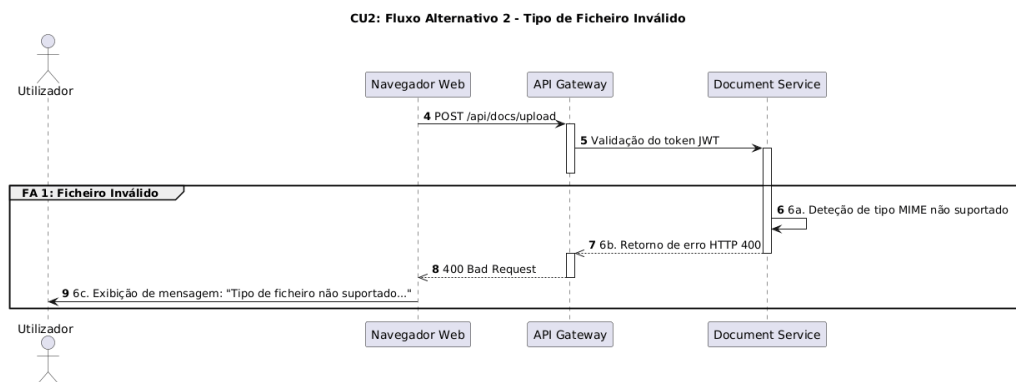


Figura 5.9: CU2 — Upload de Documento: ficheiro inválido

Fluxo Alternativo 2: Erro no Storage:

- 6a. Falha na comunicação com o *object storage*;
- 6b. Registo do erro em *logs*;
- 6c. HTTP 500 com *retry logic* (3 tentativas antes de erro definitivo).

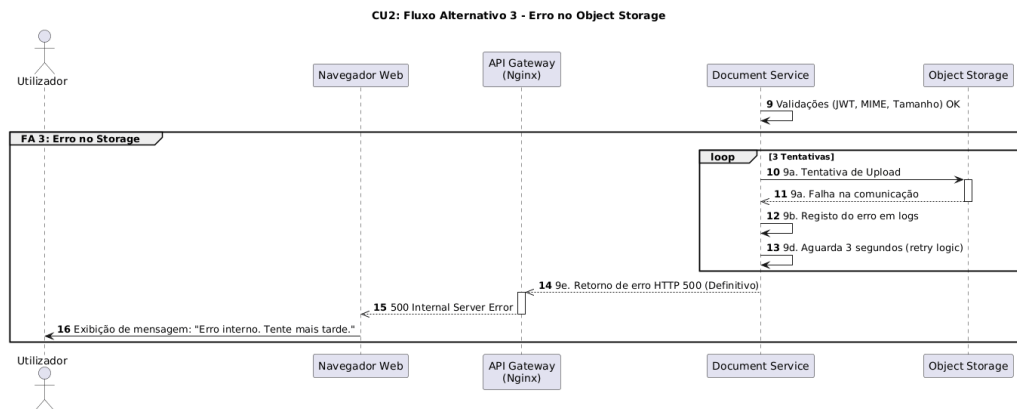


Figura 5.10: CU2 — Upload de Documento: erro no storage

Pós-condições: Documento armazenado, indexado e processado pela IA.

5.3.3 CU3: Criação de Convite para Novo Utilizador

Ator: Gestor ou Administrador

Pré-condições: Utilizador autenticado com *role* GESTOR ou ADMIN.

Fluxo Principal:

1. Acesso à secção de gestão de utilizadores;
2. Preenchimento do formulário: email do convidado e *role* a atribuir;
3. Envio de requisição `POST /api/tenants/:id/invite`;
4. Verificação de permissões (RBAC);
5. Verificação de não-existência do email;
6. Geração do código único do convite (UUID) com validade de 48 horas;
7. Publicação do evento `invite.created` no RabbitMQ;
8. O Notification Service envia email com *link* de registo;
9. Resposta HTTP 201 com confirmação.

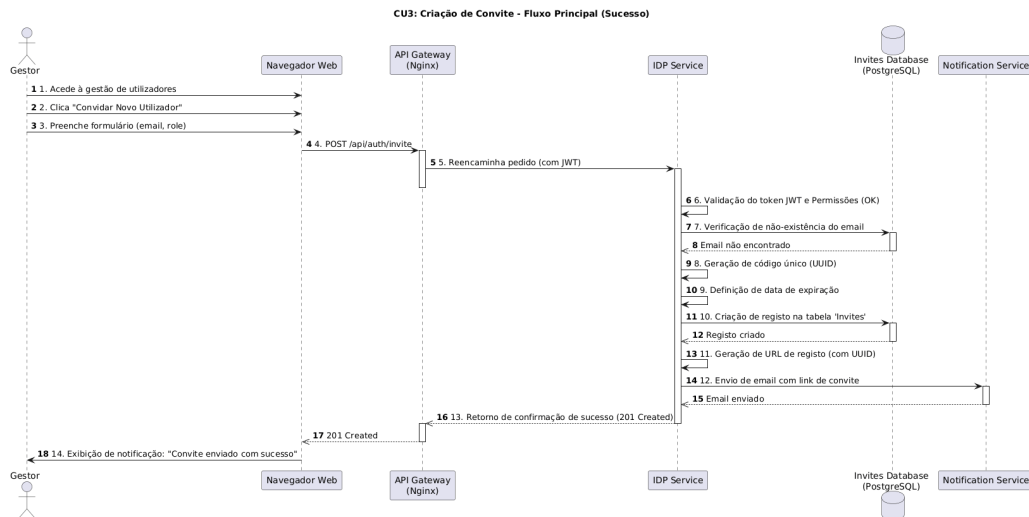


Figura 5.11: CU3 — Criação de Convite: fluxo principal (sucesso)

Fluxo Alternativo 1: Email Já Existe:

- 5a. Email já registado no sistema;
- 5b. HTTP 409: "Este email já está registado".

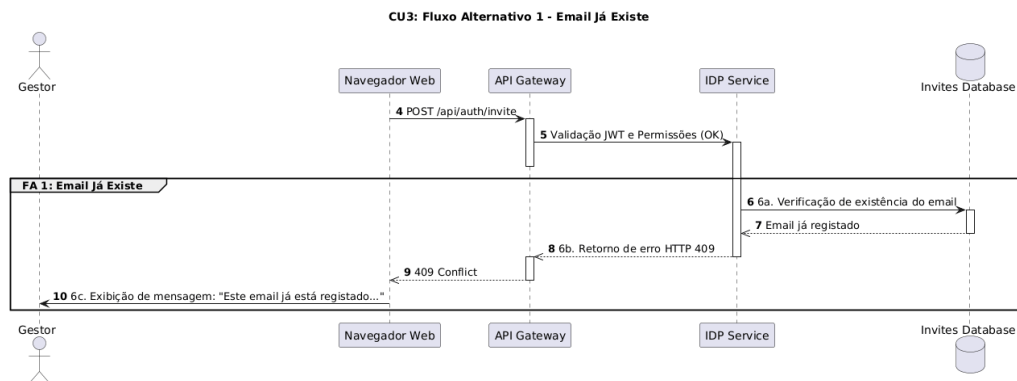


Figura 5.12: CU3 — Criação de Convite: email já existe

Fluxo Alternativo 2: Permissão Negada:

- 4a. Utilizador com *role* CLIENTE;
- 4b. HTTP 403: “Não tem permissão para criar convites”.

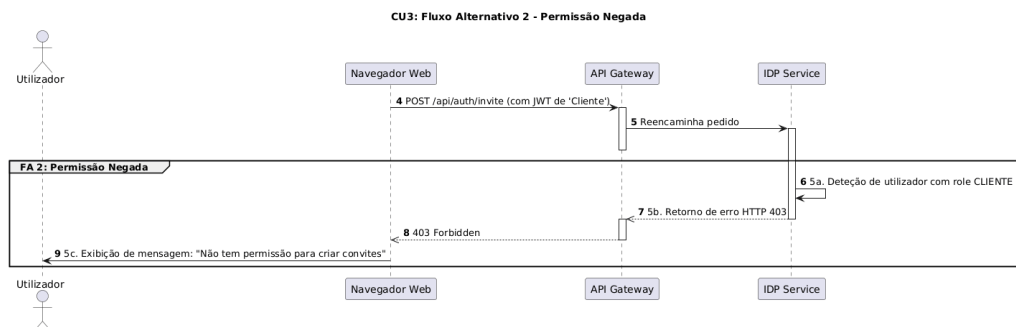


Figura 5.13: CU3 — Criação de Convite: permissão negada

Pós-condições: Convite criado na base de dados e email enviado ao destinatário.

5.4 Diagramas de Sequência

Esta secção apresenta os diagramas de sequência dos dois fluxos de interação mais relevantes da plataforma.

5.4.1 Fluxo de Autenticação (Login)

O fluxo de autenticação implementa o padrão *API Composition*: o IDP Service orquestra uma chamada ao Organization Service para obter o `organizationId` e o `role` do utilizador antes de emitir o token JWT.

1. Cliente envia credenciais via POST `/api/auth/login`;
2. API Gateway encaminha para o IDP Service;
3. IDP Service valida credenciais contra a `idp_db` (bcrypt);
4. IDP Service consulta o Organization Service para obter `organizationId` e `role`;
5. IDP Service gera o token JWT (payload: `userId`, `organizationId`, `role`);
6. Registo da ação em `AuthLog`;
7. Token retornado ao cliente (HTTP 200).

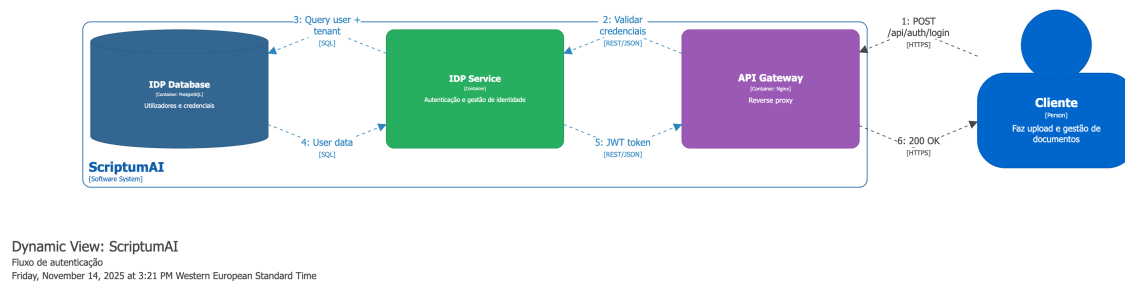
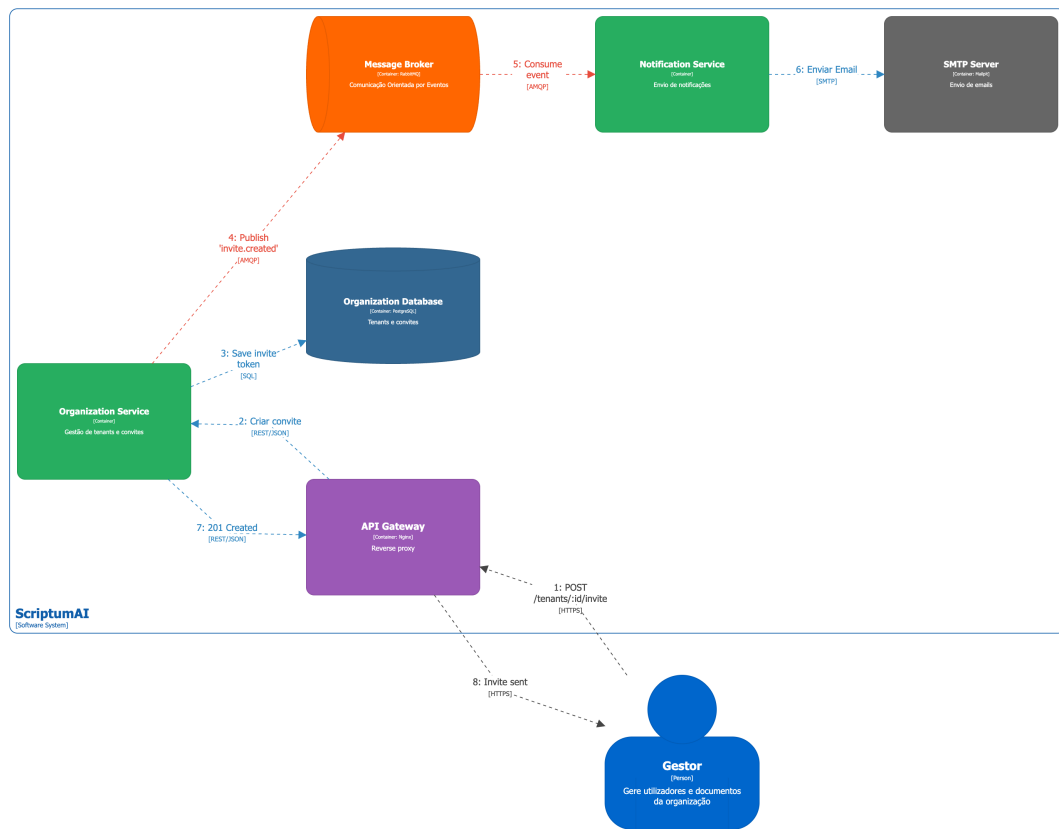


Figura 5.14: Diagrama de sequência do fluxo de autenticação

5.4.2 Fluxo de Criação de Convite

O fluxo de convite combina comunicação síncrona (validação e criação) com comunicação assíncrona (envio de email), garantindo que falhas no serviço de email não bloqueiam a operação principal.

1. Gestor envia pedido `POST /api/tenants/:id/invite`;
2. Organization Service valida permissões e verifica o email;
3. Organization Service persiste o convite em `org_db`;
4. Organization Service publica o evento `invite.created` no RabbitMQ;
5. Resposta imediata HTTP 201 ao cliente;
6. *[Assíncrono]* Notification Service consome o evento e envia email via SMTP.



Dynamic View: ScriptumAI
Fluxo de convite de utilizador para organização
Friday, November 14, 2025 at 3:21 PM Western European Standard Time

Figura 5.15: Diagrama de sequência do fluxo de criação de convite

Figura 5.16: Diagrama de sequência do fluxo de registo de utilizador

6. Arquitectura Funcional

A arquitectura funcional da plataforma é documentada segundo o modelo C4, que decompõe o sistema em níveis progressivos de detalhe, desde o contexto externo até aos componentes internos de cada serviço.

6.1 Visão Geral

O **ScriptumAI** adopta uma arquitectura de **microserviços**, em que cada serviço é independente, tem a sua própria base de dados e comunica com os demais através de interfaces bem definidas. Esta escolha garante:

- **Escalabilidade independente:** cada microserviço pode ser escalado horizontalmente sem afectar os restantes;
- **Manutenibilidade:** alterações a um serviço não requerem *redploy* de toda a plataforma;
- **Isolamento de falhas:** uma falha num serviço não provoca a queda de todo o sistema;
- **Tecnologias heterogéneas:** cada serviço pode usar a *stack* tecnológica mais adequada ao seu propósito.

Os princípios arquitecturais fundamentais são:

- **Database per Service:** cada microserviço possui e gere a sua própria base de dados;
- **API Gateway:** ponto único de entrada para todo o tráfego externo (Nginx);
- **Event-Driven:** comunicação assíncrona para operações não-bloqueantes (RabbitMQ);
- **Multi-tenancy:** isolamento de dados por `organizationId` na camada de aplicação.

6.2 Diagrama de Contexto (Nível 1)

O diagrama de contexto posiciona o **ScriptumAI** no seu ambiente externo, identificando os actores que interagem com a plataforma.

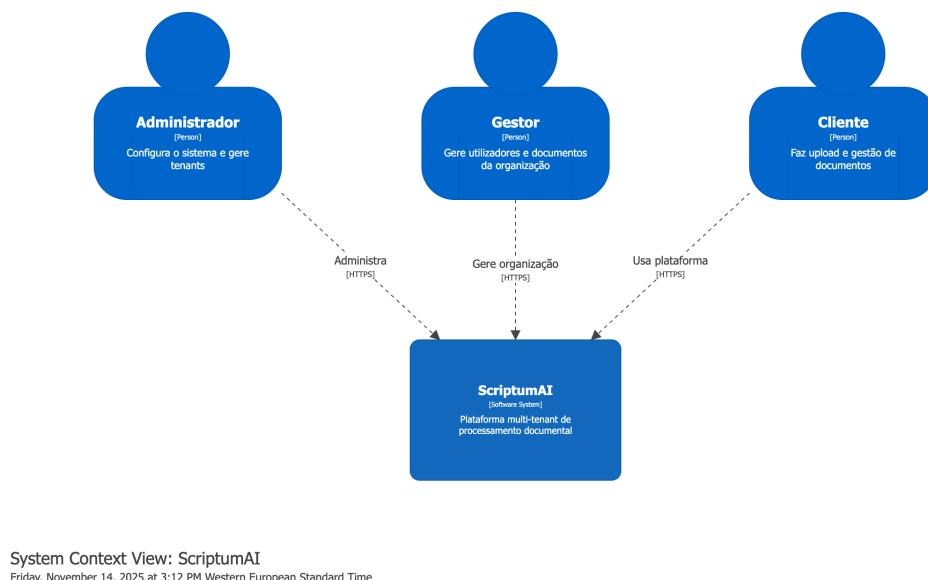


Figura 6.1: Diagrama de Contexto:Actores e Sistema ScriptumAI

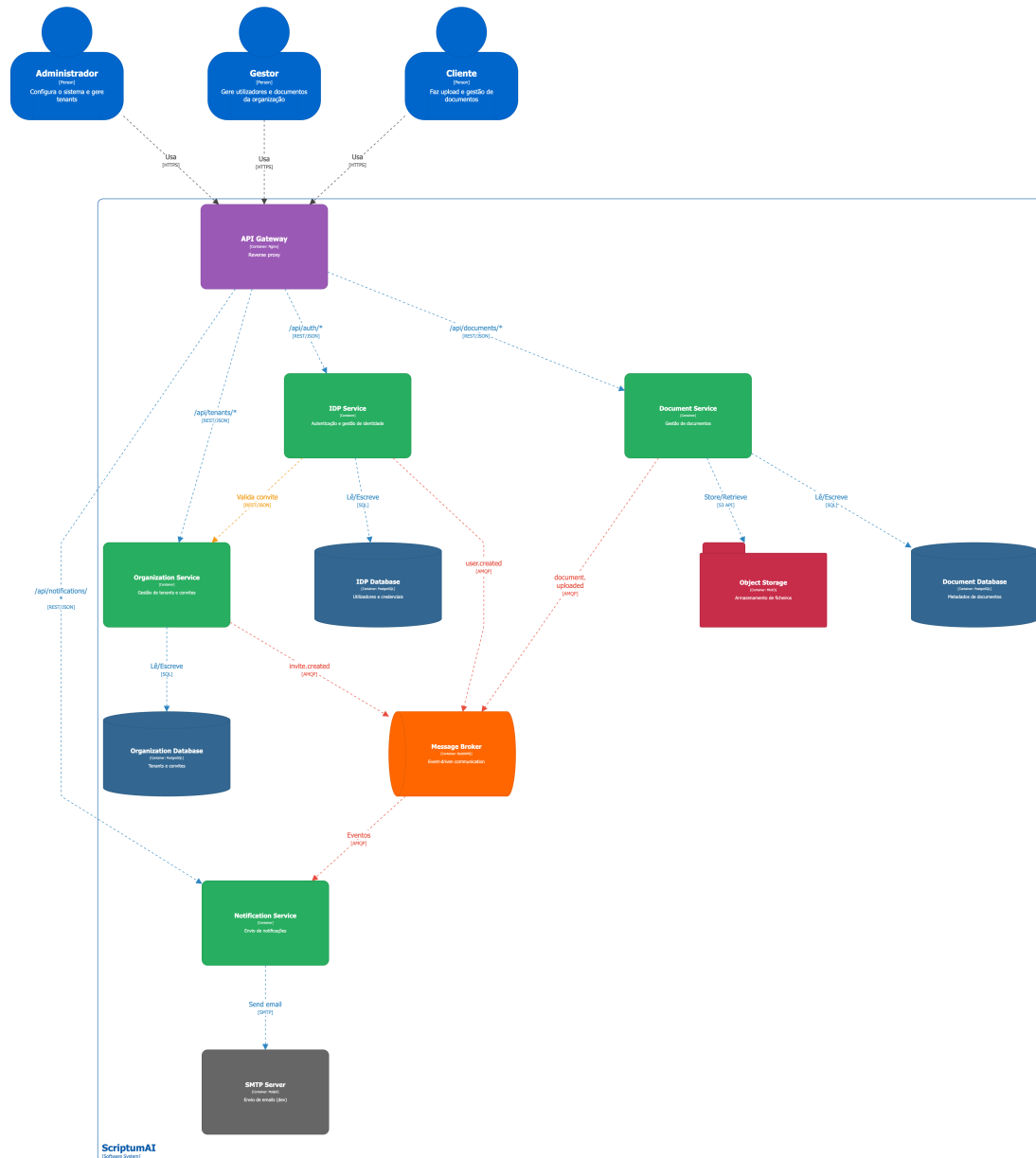
A plataforma serve três perfis de utilizadores com diferentes níveis de acesso:

- **Administrador:** configuração do sistema e gestão global da organização;
- **Gestor:** gestão de utilizadores e documentos da organização;
- **Cliente:** utilização básica: *upload*, pesquisa e consulta de documentos.

Todos os actores interagem com o sistema via HTTPS, beneficiando de autenticação JWT e controlo de acesso RBAC.

6.3 Diagrama de *Contentores* (Nível 2)

O diagrama de *contentores* revela a arquitectura interna da plataforma, mostrando os microserviços, as suas bases de dados e os serviços de infraestrutura de suporte.



Container View: ScriptumAI
Arquitetura de microserviços
Friday, November 14, 2025 at 3:12 PM Western European Standard Time

Figura 6.2: Diagrama de *Contentores*:Microserviços e Componentes

6.3.1 Microserviços

IDP Service: Provedor de Identidade

- Responsável pela gestão central da identidade dos utilizadores;
- Funções: *login/logout* (emissão de JWT), registo via convite, gestão de perfis;
- Gere a revogação de *tokens* (**TokenBlacklist**) e a auditoria de acessos (**AuthLogs**);
- *Stack*: Node.js + Express + PostgreSQL (porta 3001).

Organization Service: Gestão de Organizações

- Responsável pela lógica de multi-tenancy;
- Funções: CRUD de organizações (*tenants*), sistema de convites (**Invites**), gestão de membros e papéis (**Memberships**), gestão de departamentos (**Departments**);
- *Stack*: Node.js + Express + PostgreSQL (porta 3002).

Document Service: Gestão de Documentos

- Responsável pelas operações sobre documentos (CRUD);
- Funções: *upload* e *download* de ficheiros, integração com MinIO (*object storage*), gestão e edição de metadados (nome, descrição e departamento associado);
- *Stack*: NestJS + PostgreSQL (porta 3003).

AI Service: Processamento Inteligente

- Responsável pelo processamento de documentos com IA e pelo *chatbot* RAG;
- Funções: extração de texto, geração de resumos, *embeddings* vectoriais, pesquisa semântica e respostas por linguagem natural;
- O *chatbot* é acessível via POST `/api/documents/{documentId}/chat` e opera exclusivamente sobre os documentos do *tenant* do utilizador autenticado;
- Os *embeddings* são gerados com o modelo `paraphrase-multilingual-MiniLM-L12-v2` (HuggingFace) e armazenados em ChromaDB por documento; na consulta, os excertos mais relevantes são recuperados por similaridade semântica e enviados como contexto ao modelo de linguagem;
- O acesso ao *chatbot* é controlado por um sistema de quotas associado ao plano de subscrição: **Free Trial** (3 mensagens) e **Starter** (10 mensagens); ao atingir o limite, a API retorna HTTP 403 com o código `limit_reached`;
- *Stack*: Python + FastAPI + LangChain + ChromaDB + Ollama (Llama 3.1) + sentence-transformers.

Notification Service: Serviço de Notificações

- Responsável pelo envio de comunicações externas de forma assíncrona;
- Ouve eventos do RabbitMQ (ex.: `invite.created`) e envia emails via SMTP;
- Regista *logs* de envio (`NotificationLogs`) para auditoria;
- *Stack*: Node.js + Express (porta 3005).

6.3.2 Componentes Auxiliares (*Backing Services*)

API Gateway (Nginx) Ponto único de entrada que implementa o padrão *Reverse Proxy*. Encaminha pedidos para os microserviços correctos (ex.: `/api/auth/*` → IDP Service), realiza terminação SSL e aplica *rate limiting*.

Message Broker (RabbitMQ) Sistema de mensagens para comunicação assíncrona orientada a eventos. Implementa o padrão *publish-subscribe* para desacoplamento total entre serviços.

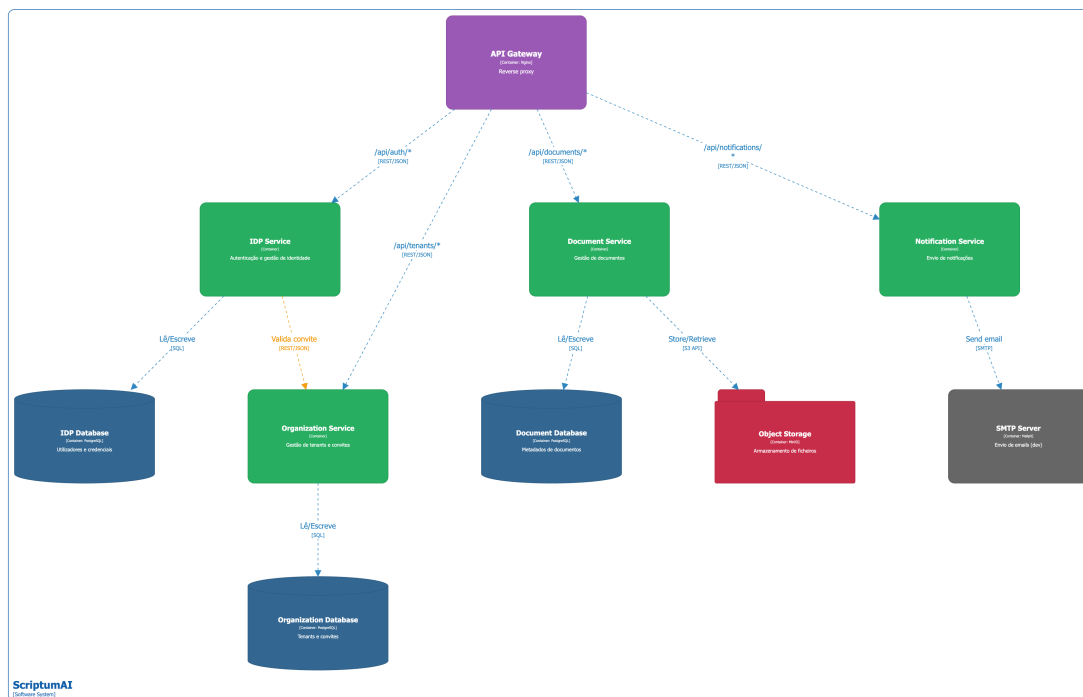
Bases de Dados (PostgreSQL) Arquitectura *Database-per-Service*: cada microserviço possui e gere a sua própria base de dados independente, garantindo isolamento de dados e escalabilidade individual.

Object Storage (MinIO) Armazenamento de ficheiros binários (PDF, DOCX, etc.) compatível com a API S3. Separa os metadados (PostgreSQL) dos ficheiros físicos.

6.4 Padrões de Comunicação

6.4.1 Comunicação Síncrona (REST/HTTP)

Utilizada para operações que requerem resposta imediata. O API Gateway valida a assinatura do JWT (usando a chave pública do IDP Service) e encaminha o pedido para o microserviço adequado. Cada microserviço valida as permissões (RBAC) contidas no *payload* do *token* sem necessidade de contactar o IDP Service, garantindo desacoplamento.



Container View: ScriptumAI
Friday, November 14, 2025 at 3:12 PM Western European Standard Time

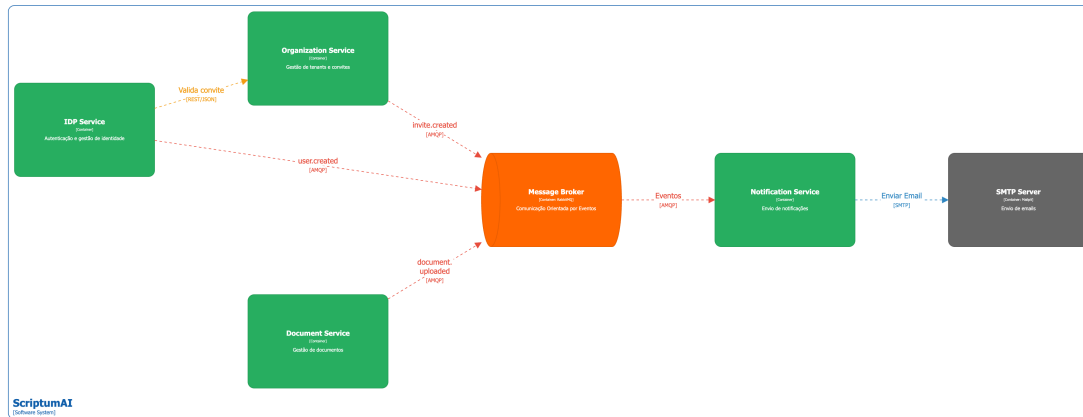
Figura 6.3: Padrão de Comunicação Síncrona via REST/HTTP

Endpoints principais:

- `/api/auth/*` → IDP Service
- `/api/tenants/*` → Organization Service
- `/api/organizations/{orgId}/departments/*` → Organization Service
- `/api/documents/*` → Document Service
- `/api/documents/{documentId}/chat` → AI Service
- `/api/ai/*` → AI Service
- `/api/notifications/*` → Notification Service

6.4.2 Comunicação Assíncrona (AMQP / RabbitMQ)

Utilizada para operações não-bloqueantes que não requerem resposta imediata. Garante que operações lentas (envio de emails, processamento de documentos por IA) não impactam a performance das requisições HTTP.



Container View: ScriptumAI
Comunicação assíncrona via RabbitMQ
Friday, November 14, 2025 at 3:23 PM Western European Standard Time

Figura 6.4: Padrão de Comunicação Assíncrona via *Message Broker*

Exemplo de fluxo assíncrono (criação de convite):

1. O Organization Service valida o pedido, persiste o convite na base de dados e publica o evento `invite.created` no RabbitMQ;
2. Retorna 201 `Created` imediatamente ao cliente sem aguardar o envio do email;
3. O Notification Service consome o evento em *background*, carrega o *template* de email, envia via SMTP e confirma o processamento (ACK ao RabbitMQ).

6.5 Arquitectura de Infraestrutura

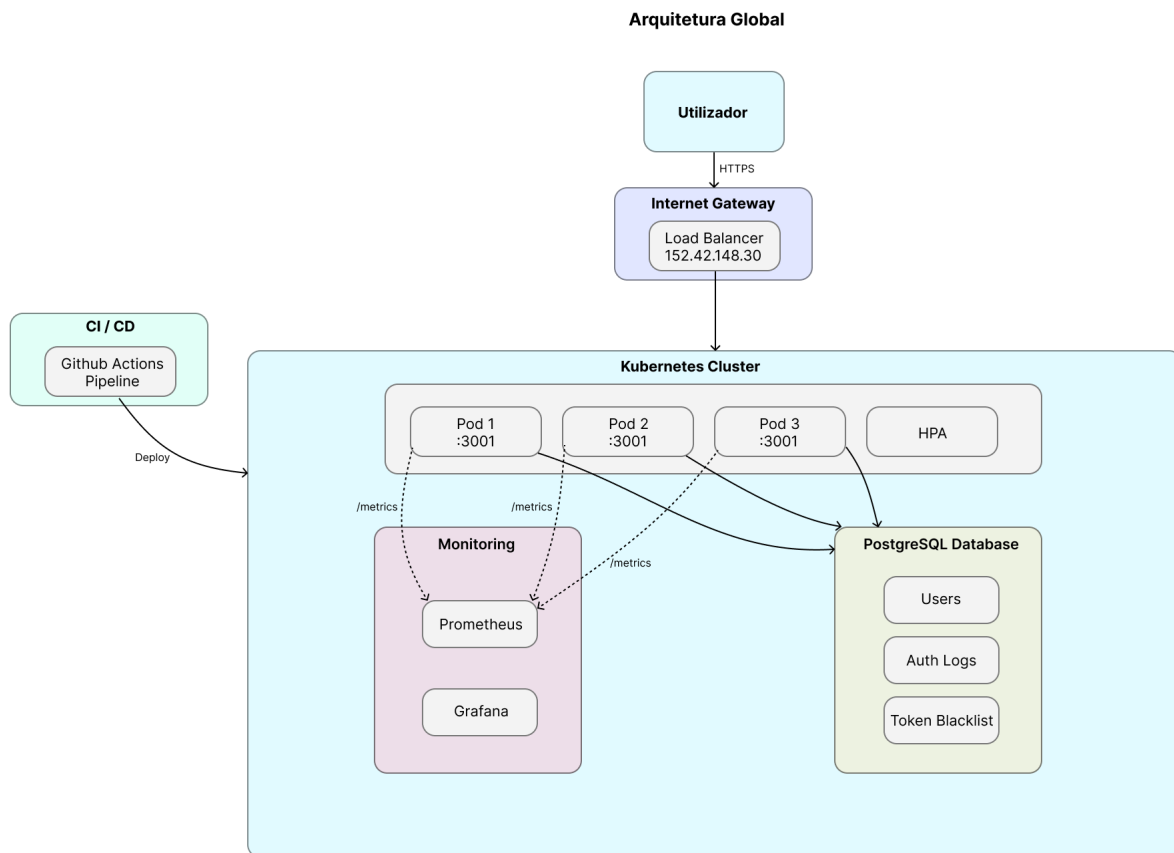


Figura 6.5: Arquitectura global de infraestrutura do sistema

A plataforma é implementada sobre Kubernetes (DigitalOcean), com os seguintes elementos de infraestrutura:

- **Load Balancer (DigitalOcean):** ponto de entrada único com IP público estático; distribui o tráfego entre os *pods* do API Gateway;
- **Cluster Kubernetes:** orquestra a execução dos microsserviços em contentores Docker, com *auto-scaling* horizontal (HPA) de 1 a 3 réplicas por serviço;
- **PostgreSQL gerido:** bases de dados persistentes com *backups* automáticos e *patching* gerido pelo fornecedor *cloud*;
- **Prometheus + Grafana:** *stack* de observabilidade para recolha de métricas e *dashboards* operacionais em tempo real;
- **GitHub Actions (CI/CD):** *pipelines* automatizadas que cobrem desde a validação de código até ao *deployment* em produção.

7. Implementação da API

A camada de *backend* foi construída como um conjunto de microserviços independentes. Nas secções seguintes apresentam-se as opções tecnológicas, os modelos de dados e o funcionamento de cada serviço.

7.1 Visão Geral da Arquitectura

O *backend* segue uma arquitectura de microserviços com comunicação assíncrona baseada em eventos. Cada serviço possui a sua própria base de dados PostgreSQL (isolamento de dados), comunica com os restantes via RabbitMQ e é exposto ao exterior através de um API Gateway NGINX. A orquestração local é feita com Docker Compose e a produção com Kubernetes.

Os serviços implementados são:

- **API Gateway:** NGINX como ponto de entrada único;
- **Authentication Service:** autenticação e gestão de utilizadores (Express.js);
- **Organization Service:** organizações, membros, convites e departamentos (NestJS + Prisma);
- **Document Service:** gestão de ficheiros e histórico de conversação RAG (NestJS + Fastify + Drizzle);
- **AI Service:** processamento de documentos e chatbot RAG (Python/FastAPI);
- **Notification Service:** notificações *push* (FCM) e e-mail (Express.js).

7.2 API Gateway

O NGINX actua como ponto de entrada único (porta 80), encaminhando pedidos para os serviços internos:

- `/api/auth/*` → *Authentication Service*;
- `/api/organizations/*`, `/api/memberships/*`, `/api/invites/*`, `/api/departments/*` → *Organization Service*;
- `/api/documents/*` → *Document Service*;
- `/api/notifications/*` → *Notification Service*;
- `/` → aplicação web (*PWA Service*).

O *gateway* está configurado com tamanho máximo de *upload* de 100 MB e *timeout* de 300 segundos para operações de longa duração (processamento IA).

7.3 Serviço de Autenticação

Desenvolvido em **Express.js** com **Sequelize** e base de dados `auth_db`, implementa:

- Registo de utilizadores e criação de organização (*onboarding* com plano de *trial*);
- *Login* com emissão de tokens JWT (acesso: 7 dias; *refresh*: 30 dias);
- Renovação de token via *endpoint* `/refresh`;
- Recuperação e redefinição de *password* com token temporário (`PasswordResetToken`);
- *Logout* com invalidação de token (`TokenBlacklist`);
- Registo de auditoria de autenticação (`AuthLog`) com IP e *user agent*;
- Gestão de perfil e alteração de *role* por administrador.

O modelo `User` armazena identificador UUID, e-mail, *password* com *hash* `bcrypt`, *username*, nome, apelido, *role* (`user/admin`) e estado (`active/inactive`).

7.4 Serviço de Organização

Desenvolvido em **NestJS** com **Prisma** e base de dados `organization_db`, gere o modelo multi-*tenant* da plataforma:

Modelos de Dados

- **Organization:** nome, e-mail, subdomínio, cidade, morada, contacto, código postal e estado (*active/inactive/pending/suspended*);
- **Membership:** associação entre utilizador e organização com *role* (*ADMIN/USER*);
- **Invite:** convite por e-mail com token UUID, *role* inicial e data de expiração;
- **Department:** departamentos definidos pela organização (nome e descrição);
- **DepartmentMembership:** associação entre utilizador e departamento, com par único;
- **Subscription:** plano e estado de subscrição (*trialing/active/expired*) com datas de início e fim de *trial*.

Funcionalidades

O serviço expõe *endpoints* CRUD para organizações, membros, convites e departamentos. O *endpoint* `/api/organizations/onboarding-trial` cria, numa única chamada, a organização, o utilizador administrador e a subscrição de *trial*. A suspensão/reactivação de organizações é suportada por acções de *staff*.

7.5 Serviço de Documentos

Desenvolvido em **NestJS** com adaptador **Fastify** e ORM **Drizzle**, utiliza PostgreSQL (`document_db`) para metadados e **MinIO** para armazenamento de ficheiros.

Modelos de Dados

- **documents:** metadados do ficheiro (nome, tipo MIME, tamanho, *bucket*, chave de objeto, departamento, descrição) e resultados IA (estado, resumo, palavras-chave, entidades e confiança);
- **chatMessages:** histórico de conversação RAG por documento (papel, conteúdo, utilizador, organização);
- **globalChatMessages:** histórico do assistente global com lista de documentos fonte em JSONB.

Funcionalidades

O serviço suporta *upload* via *multipart/form-data* (ficheiros binários) e JSON (documentos de texto). Os formatos aceites são PDF, DOCX, TXT, JPG e PNG. Os ficheiros são armazenados no MinIO sob o caminho `/documents/{orgId}/{docId}/{filename}`.

Após o *upload*, é publicado o evento `document.uploaded` no RabbitMQ. Quando o AI Service conclui o processamento, o serviço recebe o evento `document.processed` e actualiza os campos IA no registo.

Os *endpoints* de chat (`/api/documents/{id}/chat` e `/api/documents/global-chat`) encaminham as mensagens para o AI Service e persistem o histórico.

7.6 Serviço de IA

Desenvolvido em **Python** com **FastAPI** e **Uvicorn**, é responsável pelo processamento de documentos e pelo chatbot RAG.

Pipeline de Processamento

O serviço consome eventos `document.uploaded` do RabbitMQ e executa o seguinte fluxo:

1. Transferência do ficheiro a partir do MinIO;
2. Extração de texto conforme o tipo: PyPDF para PDF (com OCR via Tesseract para páginas digitalizadas), python-docx para DOCX e leitura directa para TXT e imagens;
3. Divisão em fragmentos com `RecursiveCharacterTextSplitter` (Langchain);
4. Geração de *embeddings* com o modelo **paraphrase-multilingual-MiniLM-L12-v2** (suporte a português e inglês);
5. Armazenamento de vectores no **ChromaDB**, isolado por organização em directório próprio;
6. Geração de resumo e extração de palavras-chave via LLM local (**Llama 3.1** via Ollama);
7. Publicação do evento `document.processed` com os resultados IA.

Chatbot RAG

O serviço expõe dois *endpoints* de conversação:

- POST `/documents/{document_id}/chat`: pesquisa de similaridade restrita ao documento especificado (metadados ChromaDB por `document_id`), com contexto injectado no *prompt* para o LLM;

- **POST /documents/global-chat**: pesquisa de similaridade nos documentos da organização (isolamento por `organization_id`), com devolução dos documentos fonte e respectiva pontuação de relevância.

O LLM é executado localmente com temperatura 0 (respostas determinísticas) e *timeout* de 120 segundos. O cliente de chatbot no Document Service usa um *timeout* de 300 segundos para acomodar a latência do modelo.

7.7 Serviço de Notificações

Desenvolvido em **Express.js** com **Firebase Admin SDK** e **Nodemailer**, gere notificações *push* e e-mail.

- **Notificações *push* (FCM)**: registo e cancelamento de tokens de dispositivo; envio de alertas de processamento concluído e partilha de documentos;
- **E-mail**: envio via SMTP com templates **Handlebars** (boas-vindas, convite, resumo IA);
- **Registo de notificações**: histórico de notificações por utilizador com estado (SENT/FAILED), filtros por destinatário e acções de marcação como lida, eliminação individual e limpeza total;
- **Formulário de contacto**: recepção e armazenamento de mensagens de suporte.

O serviço consome eventos RabbitMQ: `document.ai.completed` (notificação ao utilizador), `user.invited` (e-mail de convite) e `user.registered` (e-mail de boas-vindas).

7.8 Comunicação entre Serviços

A comunicação assíncrona é centralizada no *broker* **RabbitMQ** com uma *exchange* do tipo *topic* (`scriptumai.events`). O fluxo principal é o seguinte:

1. O *Document Service* publica `document.uploaded` após armazenar o ficheiro;
2. O *AI Service* consome o evento, processa o documento e publica os eventos de conclusão e notificação;
3. O *Document Service* actualiza os metadados IA ao receber `document.processed`;
4. O *Notification Service* envia a notificação ao utilizador ao receber o evento de conclusão IA.

A comunicação síncrona HTTP é usada para operações pontuais: o *Authentication Service* invoca o *Organization Service* para criar a organização no registo, e o *Document Service* invoca o *AI Service* para pedidos de chat.

7.9 Armazenamento e Infraestrutura

- **PostgreSQL 16:** base de dados separada por serviço (`auth_db`, `organization_db`, `document_db`, `notification_db`), com utilizador e credenciais próprios por base de dados;
- **MinIO:** armazenamento de objetos compatível com S3 para ficheiros de documentos;
- **ChromaDB:** base de dados vectorial persistente para *embeddings* RAG, isolada por organização;
- **Docker Compose:** orquestração local com ordem de arranque controlada e *health-checks*;
- **Kubernetes:** manifestos para produção com *deployments*, *services*, *ConfigMaps*, limites de recursos, *liveness/readiness probes* e *Horizontal Pod Autoscaler* (HPA).

8. Implementação da Aplicação Web

A aplicação web foi desenvolvida com foco na experiência de administração e na gestão documental multi-*tenant*. As secções seguintes detalham as tecnologias adoptadas, a estrutura do projecto e as funcionalidades implementadas.

8.1 Tecnologias Utilizadas

A aplicação web foi desenvolvida com as seguintes tecnologias:

- **Next.js 16:** *framework* React com *App Router*, *Server Components* e geração de páginas estática e dinâmica;
- **React 19:** biblioteca de construção de interfaces com suporte a *Concurrent Features*;
- **TypeScript:** tipagem estática para segurança e manutenibilidade do código;
- **Tailwind CSS v4:** *framework* de estilos utilitário para interfaces responsivas;
- **shadcn/ui:** componentes reutilizáveis baseados em Radix UI e Tailwind CSS;
- **Recharts:** biblioteca de gráficos para visualização de dados;
- **Lucide React:** conjunto de ícones SVG.

8.2 Estrutura do Projeto

A aplicação segue a convenção de *App Router* do Next.js com separação de rotas por grupos:

- **src/app/(auth)/:** rotas públicas: *login*, registo, recuperação e redefinição de *password*, aceitação de convite, *trial* expirado;
- **src/app/(app)/:** rotas protegidas: *dashboard*, administração, departamentos, convites, equipa e secção de *staff*;
- **src/components/:** componentes partilhados (navegação, diálogos, UI genérica);
- **src/services/:** camada de acesso à API REST;

- `src/context/`: contextos React para estado global;
- `src/lib/`: utilitários auxiliares.

8.3 Autenticação e Gestão de Sessão

A autenticação é gerida com tokens JWT armazenados em *cookies* seguros. O fluxo inclui dois caminhos de registo distintos:

- **Registo de organização**: formulário por etapas com selecção de plano de subscrição, dados pessoais (nome, apelido, *username*), dados da organização (nome, cidade, morada, contacto, código postal) e credenciais de acesso;
- **Registo por convite**: o utilizador acede a um URL com código UUID único e completa o registo sem necessidade de seleccionar plano;
- **Login**: submissão de credenciais com renovação automática de tokens em respostas HTTP 401;
- **Recuperação de *password***: pedido de *e-mail* de recuperação e redefinição com token temporário.

Quando o período de *trial* expira, o utilizador é reencaminhado para `/trial-expired` com indicação da data de término e opções de *upgrade*.

8.4 Funcionalidades Implementadas

8.4.1 *Dashboard*

A página principal apresenta uma visão consolidada da organização:

- **Cabeçalho**: nome e *e-mail* do utilizador com indicador de *role* e *banner* de *trial* activo;
- **KPIs**: total de documentos, percentagem de documentos processados por IA (gráfico *donut*), documentos IA pendentes, número de convites activos e utilização de armazenamento;
- **Gráfico de tipos de documentos**: distribuição de PDF, imagens, texto e outros (gráfico de barras);
- **Cartão de organização**: nome, *e-mail*, cidade, morada, contacto, código postal, estado e data de criação, com lista de departamentos;
- **Gestão de membros da organização** (Administrador/*Staff*): lista de membros pesquisável com atribuição de departamentos;
- **Documentos recentes**: tabela com ícone de ficheiro, nome, tamanho, *badge* do utilizador que carregou, estado de processamento IA e data.

8.4.2 Gestão de Membros

A página `/admin/members` (acessível a Gestores e Administradores) disponibiliza:

- KPIs de membros: total, administradores e utilizadores regulares;
- Tabela pesquisável com nome, *e-mail*, *role* (alterável via *dropdown*), estado e data de entrada;
- Remoção de membros com confirmação; o utilizador não pode alterar o seu próprio *role*.

8.4.3 Gestão de Convites

A página `/admin/invites` permite:

- Visualizar KPIs: total, pendentes e expirados;
- Criar convites com definição de *role* inicial;
- Listar convites com estado (*Pending/Expired*), *e-mail*, *role* e tempo restante;
- Reenviar ou revogar convites individualmente.

8.4.4 Gestão de Departamentos

A página `/departments` inclui:

- KPIs: total de departamentos e total de atribuições de membros;
- Criação e edição de departamentos (nome e descrição);
- Lista expansível com contagem de membros, edição e eliminação;
- Diálogo de adição de membros com pesquisa.

8.4.5 Secção de *Staff*

A secção em `/admin` e `/staff` é exclusivo para utilizadores com perfil de *staff* da plataforma e apresenta:

- **Visão geral:** KPIs de utilizadores (total, administradores, regulares, organizações) e gráfico de utilizadores por *role*;
- **Utilizadores:** tabela pesquisável e filtrável (por *role*, estado, organização) com linhas expansíveis mostrando ID, *username* e data de criação;
- **Conteúdo:** KPIs de documentos (total, processados, pendentes, falhados), distribuição por tipo, contagem de utilizadores por organização e tabela completa de documentos;

- **Registos de actividade:** histórico de *uploads* com avatar, nome do documento, *timestamp*, tamanho e estado IA;
- **Organizações** (`/staff/organizations`): listagem de todas as organizações com estado (Activa, Suspensa, Pendente), plano e acções de suspensão/reactivação com confirmação;
- **Detalhe de organização** (`/staff/organizations/[id]`): vista individual de cada organização com gestão de membros (edição de *role*, remoção) e gestão de convites (criação, cancelamento) no contexto específico dessa organização.

8.5 Comunicação com o *Backend*

Toda a comunicação é centralizada na camada de serviços, que define os tipos TypeScript para todos os modelos de dados, encapsula as chamadas HTTP com tratamento de erros tipado (`ApiRequestError`) e lê a variável de ambiente `NEXT_PUBLIC_API_URL` para suportar múltiplos ambientes.

8.6 Responsividade

A aplicação foi concebida com *design* responsivo, adaptando-se a diferentes resoluções de ecrã. O *service worker* gerado pelo Next.js permite a instalação no dispositivo e funcionamento em modo *offline* para páginas previamente visitadas.

9. Implementação da Aplicação Móvel

A aplicação móvel é uma aplicação nativa Android que disponibiliza ao utilizador final as funcionalidades de gestão documental e de assistente conversacional. De seguida descrevem-se a arquitectura, as tecnologias e os ecrãs implementados.

9.1 Tecnologias Utilizadas

A aplicação Android foi desenvolvida com as seguintes tecnologias:

- **Kotlin**: linguagem principal, com suporte a *coroutines* para programação assíncrona;
- **Jetpack Compose**: *toolkit* declarativo de construção de interfaces nativas (Material Design 3);
- **Retrofit 2 + OkHttp 4**: cliente HTTP com interceptores para injeção de tokens e renovação automática;
- **Room**: base de dados SQLite local para persistência de documentos em *cache*;
- **Firebase Cloud Messaging (FCM)**: notificações *push* para alertas de processamento e partilha;
- **Google ML Kit Document Scanner**: digitalização de documentos físicos via câmara;
- **DataStore**: armazenamento reactivo de preferências e dados de sessão.

9.2 Arquitectura

A aplicação segue o padrão **MVVM** com organização em três camadas:

- **Dados (data/)**: modelos, clientes de rede (Retrofit), repositórios e base de dados Room;
- **Domínio (domain/)**: casos de uso que encapsulam a lógica de negócio;
- **Apresentação (presentation/)**: *ViewModels* com **StateFlow** e ecrãs Compose.

9.3 Camada de Rede

O `RetrofitClient` configura dois clientes HTTP: um padrão (leitura: 30 s) para todas as operações e um cliente de chatbot (leitura: 300 s) para suportar a latência do LLM. Ambos partilham o `TokenInterceptor` (injeta o token JWT em todos os pedidos) e o `TokenAuthenticator` (renova o token em respostas HTTP 401; em caso de falha, invalida a sessão e redirige para *login*). O cliente detecta automaticamente execução em emulador e ajusta o URL base.

9.4 Casos de Uso

A camada de domínio implementa os seguintes casos de uso:

- `LoginUseCase`, `RegisterUseCase`: autenticação e registo;
- `ForgotPasswordUseCase`, `ResetPasswordUseCase`: recuperação de *password*;
- `ListDocumentsUseCase`, `UpdateDocumentUseCase`, `DeleteDocumentUseCase`: gestão de documentos;
- `ListDepartmentsUseCase`: listagem de departamentos da organização;
- `GetDocumentChatHistoryUseCase`, `GetDocumentChatUsageUseCase`: histórico e quota do chat por documento;
- `GetGlobalChatHistoryUseCase`, `GetGlobalChatUsageUseCase`: histórico e quota do assistente global;
- `RegisterDeviceUseCase`, `UnregisterDeviceUseCase`: registo e cancelamento de dispositivo para notificações FCM;
- `MarkAllReadUseCase`, `DeleteLogUseCase`, `ClearAllLogsUseCase`: gestão de notificações.

9.5 Funcionalidades Implementadas

9.5.1 Autenticação

O fluxo de autenticação inclui:

- ***Splash Screen***: logótipo centrado com validação de sessão activa antes de navegar;
- ***Login***: campos de *e-mail* e *password* com *toggle* de visibilidade, ligação para recuperação de *password* e indicador de carregamento;
- ***Registo***: selecção de plano, dados pessoais (nome, apelido, *username*), dados de organização (nome, cidade, morada, contacto, código postal), *e-mail*, *password* e aceitação de termos; planos pagos apresentam aviso e desactivam o botão de registo;

- **Planos de subscrição** (`PricingScreen`): cartões com nome, preço, descrição e lista de funcionalidades, com destaque para o plano mais popular;
- **Recuperação de *password***: envio de *e-mail* com ecrã de confirmação de envio.

9.5.2 Navegação Principal

O `HomeScreen` é o centro de navegação da aplicação:

- **Barra de navegação inferior**: quatro separadores: Início, Documentos, Chat Global e *Upload*;
- **Drawer lateral**: perfil do utilizador (nome, iniciais e *e-mail*), *toggle* de notificações, selector de tema (Claro/Escuro/Sistema), selector de idioma (PT/EN), e acesso a Definições, Ajuda e *Logout*;
- **Separador Início**: saudação com nome do utilizador, cartões de estatísticas (total de documentos, processados por IA, pendentes) e lista de documentos recentes com estado IA.

9.5.3 Gestão de Documentos

O `DocumentsScreen` apresenta:

- Barra de pesquisa e filtros por departamento (deslocamento horizontal);
- Cartões de documento com nome, departamento, tipo de ficheiro, data, estado IA, tamanho e botões de edição/eliminação;
- Ordenação por data (crescente/decrescente), nome (A-Z/Z-A) e tamanho;
- *Pull-to-refresh* e estados de vazio/erro com botão de nova tentativa.

O `AddDocumentScreen` (separador *Upload*) suporta selecção de ficheiro nativo ou digitalização via câmara (ML Kit), com campos de nome, descrição e departamento. O progresso é apresentado em quatro estados distintos: a carregar, em processamento IA (com passos), concluído e erro com opção de nova tentativa.

9.5.4 Detalhe do Documento e Chatbot RAG

O `DocumentDetailScreen` apresenta informação completa do documento e integra um assistente conversacional por documento:

- **Cabeçalho:** nome, tipo de ficheiro, estado IA e departamento;
- **Resumo IA:** texto expansível com indicador de leitura e *tags* de palavras-chave;
- **Metadados:** tipo, tamanho, data de *upload*, utilizador, departamento e descrição;
- **Chat RAG por documento:** disponível após processamento IA concluído; apresenta o número de questões restantes (por quota de plano), histórico de mensagens com bolhas diferenciadas utilizador/IA, indicação dos documentos fonte com pontuação de relevância, e campo de envio desactivado ao atingir a quota;
- **Acções:** visualizar, descarregar e partilhar documento.

9.5.5 Assistente de IA Global

O `GlobalChatScreen` implementa um assistente conversacional de âmbito geral, não restrito a um documento específico. Apresenta o número de mensagens restantes, histórico com bolhas diferenciadas e, nas respostas da IA, indica os documentos fonte com a respectiva pontuação de relevância. O campo de envio é desactivado ao atingir a quota.

9.5.6 Notificações *Push*

O `ScriptumFirebaseMessagingService` trata notificações de processamento concluído e de partilha de documentos. No arranque da aplicação, o token FCM é registado no servidor (`registerDevice`); ao terminar sessão, o dispositivo é removido (`unregisterDevice`), garantindo que notificações não são entregues a sessões inactivas.

O `NotificationsScreen` apresenta a lista de notificações com separadores *Todas* e *Não lidas*, com as seguintes acções disponíveis:

- Marcação individual como lida;
- Marcação de todas como lidas;
- Eliminação de notificação individual;
- Limpeza de todas as notificações;
- Atualização via *pull-to-refresh*.

9.5.7 Perfil e Suporte

- **ProfileScreen:** dados pessoais, organização, plano/estado de *trial*, *role* e data de criação da conta;

- **EditProfileScreen**: edição de nome, apelido e *username* com validação e diálogo de confirmação automático;
- **ChangePasswordScreen**: alteração de *password* com validação de correspondência e comprimento mínimo;
- **HelpDeskScreen**: oito perguntas frequentes pesquisáveis com respostas expansíveis;
- **ContactUsScreen**: mapa Google Maps com localização da empresa e formulário de contacto (nome, *e-mail*, telefone, assunto e mensagem) com confirmação de envio.

9.6 Interface e Experiência do Utilizador

A interface foi construída com **Jetpack Compose** seguindo as directrizes do **Material Design 3**. O tema suporta modo claro, escuro e automático (sistema), configurável pelo utilizador no drawer. A navegação é gerida pelo **NavHost** com rotas definidas em **MainActivity**.

Manual de Utilização

O manual de utilização detalhado da aplicação móvel encontra-se em anexo.

10. Propósitos e Objetivos Cumpridos

Nesta secção avalia-se o grau de cumprimento dos propósitos e objetivos propostos inicialmente, confrontando a especificação com o que foi efetivamente implementado.

10.1 Análise de Cumprimento dos Objetivos

A tabela seguinte resume o estado de cada objetivo estratégico proposto.

Objetivo Proposto	Estado	Evidência
Plataforma de gestão documental com processamento inteligente por IA para PMEs	✓ Cumprido	AI Service com extração de texto, classificação, resumo e <i>embeddings</i>
Redução do tempo de pesquisa e classificação de documentos	✓ Cumprido	Pesquisa por texto completo, filtros por departamento/-data, chatbot RAG
Modelo de subscrição SaaS com preços escaláveis	✓ Cumprido	5 planos implementados: Free Trial, Starter, Base, Profissional, Empresarial
Conformidade com RGPD e isolamento rigoroso entre <i>tenants</i>	✓ Cumprido	Multi-tenancy por <code>organizationId</code> , JWT+RBAC, HTTPS/-TLS, bcrypt
Arquitectura escalável e disponível	✓ Cumprido	Kubernetes com HPA (1–3 réplicas), <i>rolling updates</i> , <i>health checks</i>
Aplicação móvel nativa (Android)	✓ Cumprido	App Android em Kotlin com MVVM, Retrofit, Room, FCM
Assistente conversacional RAG por documento	✓ Cumprido	Chatbot via POST <code>/api/documents/{id}/chat</code> com ChromaDB + Llama 3.1
Controlo de acesso por perfis (RBAC)	✓ Cumprido	3 perfis: Cliente, Gestor, Administrador
Sistema de convites para registo controlado	✓ Cumprido	Convites com código UUID e validade de 48 horas
Notificações <i>push</i> e email	✓ Cumprido	FCM (Firebase Cloud Messaging) + SMTP assíncrono via RabbitMQ

(continua na página seguinte)

Objetivo Proposto	Estado	Evidência
Pipeline CI/CD automatizada	✓ Cumprido	GitHub Actions com <i>build</i> , <i>test</i> , <i>deploy</i> em Kubernetes
Crescimento para novos sectores verticais	✓ Cumprido	Arquitetura multi-tenant genérica; organizações configuraram os seus próprios departamentos

Tabela 10.1: Estado de cumprimento dos objetivos propostos

10.2 Funcionalidades Implementadas

10.2.1 Aplicação Web (PWA)

A aplicação web foi desenvolvida em **Next.js** com as seguintes funcionalidades:

- **Autenticação:** *login*, registo por convite, recuperação de *password* e *refresh* automático de tokens JWT;
- **Dashboard:** listagem e pesquisa de documentos com filtros por departamento;
- **Gestão de Documentos:** *upload* com processamento IA, visualização de resumo e palavras-chave, edição de metadados (nome, descrição, departamento) e eliminação;
- **Chatbot RAG:** interface conversacional por documento com controlo de quotas por plano;
- **Gestão de Utilizadores:** área de administração com listagem de membros, edição de *roles* e gestão de convites;
- **Gestão de Departamentos:** criação, edição e eliminação de departamentos por organização;
- **Distribuição:** todos os componentes contentorizados em Docker, orquestrados em Kubernetes.

10.2.2 Aplicação Móvel (Android)

A aplicação Android foi desenvolvida em **Kotlin** com arquitectura **MVVM** e as seguintes funcionalidades:

- **Autenticação:** *login*, registo por convite e recuperação de *password*; gestão de tokens JWT com `TokenManager` e `TokenAuthenticator` para renovação automática;
- **Gestão de Documentos:** listagem com filtros por departamento, *upload* de ficheiros, visualização de detalhes (resumo, palavras-chave), edição de metadados e eliminação;

- **Chatbot Global:** assistente conversacional de âmbito geral (não por documento);
- **Notificações *Push*:** integração com Firebase Cloud Messaging (FCM) para alertas de processamento e partilha de documentos;
- **Perfil de Utilizador:** visualização e edição de dados pessoais;
- **Administração:** gestão de utilizadores e convites para perfis com permissões de Gestor/Admin;
- **Base de Dados Local:** armazenamento de tokens e preferências com `DataStore`.

10.2.3 Servidor e Infraestrutura

- **5 microsserviços** implementados e em produção: IDP Service, Organization Service, Document Service, AI Service e Notification Service;
- **API Gateway** em Nginx com *rate limiting*, terminação SSL e encaminhamento para microsserviços;
- **Pipeline de IA:** extração de texto, geração de resumo, extração de palavras-chave e criação de *embeddings* vectoriais para RAG;
- **Observabilidade:** Prometheus + Grafana com *dashboards* operacionais e alertas;

10.3 Requisitos Avançados Implementados

Para além dos requisitos base definidos no enunciado, o grupo implementou os seguintes requisitos avançados:

- **Arquitetura de microsserviços** com comunicação síncrona (REST) e assíncrona (RabbitMQ/AMQP);
- **Processamento de IA** com LLMs locais (Llama 3.1 via Ollama) e *embeddings* multilingues;
- **Chatbot RAG** com isolamento total por *tenant*;
- **Testes automatizados** com cobertura de testes unitários, de integração e E2E;
- **CI/CD** com GitHub Actions;
- **Notificações push** via Firebase Cloud Messaging na aplicação móvel;
- **Documentação OpenAPI** para todas as APIs REST;

11. Testes Realizados

Os testes funcionais e de desempenho realizados às aplicações foram organizados em quatro categorias: testes unitários, testes de integração, testes *end-to-end* (E2E) e testes de carga e desempenho.

11.1 Testes Unitários

Os testes unitários validam o comportamento isolado de cada módulo, utilizando *mocks* para substituir dependências externas. Foram implementados com a *framework* **Jest** nos serviços Node.js.

11.1.1 Authentication Service

O serviço de autenticação possui a maior cobertura de testes unitários, abrangendo os seguintes módulos:

Suite de Testes	Casos Cobertos
<code>authService.test.js</code>	Registo de utilizador, login com credenciais válidas e inválidas, logout com invalidação de token, renovação de sessão por <i>refresh token</i>
<code>tokenService.test.js</code>	Geração de tokens JWT, verificação de assinatura, deteção de token expirado, gestão da <i>blacklist</i>
<code>authMiddleware.test.js</code>	Validação de token em pedidos HTTP, rejeição de tokens inválidos ou expirados, extração de <i>claims</i> (<code>userId</code> , <code>organizationId</code> , <code>role</code>)
<code>validation.test.js</code>	Regras de validação de <i>password</i> (mínimo 8 caracteres, maiúsculas, minúsculas, números e caracteres especiais), validação de formato de email
<code>errorHandler.test.js</code>	Respostas de erro padronizadas (HTTP 400, 401, 403, 404, 409, 500), mensagens de erro localizadas
<code>requestUtils.test.js</code>	Extração de IP, <i>User-Agent</i> e <i>headers</i> de pedidos HTTP

Tabela 11.1: Suites de testes unitários do Authentication Service

11.1.2 Document Service

O Document Service inclui testes unitários da lógica de negócio principal:

- `document.service.spec.ts`: Testa as operações CRUD de documentos, validação de *ownership* por `organizationId` e `userId`, lógica de filtragem por departamento e geração de metadados.

11.2 Testes de Integração

Os testes de integração verificam o comportamento do sistema com as dependências reais (base de dados de teste, *object storage* local).

11.2.1 Authentication Service

- `authRoutes.test.js`: Testa os *endpoints* HTTP completos do serviço de autenticação contra uma base de dados PostgreSQL de teste. Cobre os fluxos de *login*, registo, *logout* e renovação de token com validação das respostas HTTP e do estado persistido.

11.3 Testes *End-to-End*

Os testes E2E validam fluxos completos do sistema, cobrindo múltiplos serviços em conjunto. Foram implementados com **Jest** e **Supertest** no Organization Service.

Ficheiro de Teste	Fluxo Coberto
<code>organizations.e2e-spec.ts</code>	Criação, leitura, atualização e eliminação de organizações; validação de unicidade de <i>subdomain</i> ; verificação de isolamento de dados entre <i>tenants</i>
<code>invites.e2e-spec.ts</code>	Criação de convite, rejeição por email duplicado, expiração de convite, aceitação de convite e marcação como utilizado
<code>memberships.e2e-spec.ts</code>	Adição de membro a organização, alteração de <i>role</i> , remoção de membro, validação de permissões por <i>role</i>
<code>full-flow.e2e-spec.ts</code>	Fluxo completo: criação de organização → convite → registo de utilizador → login → upload de documento → verificação de isolamento

Tabela 11.2: Testes E2E do Organization Service

12. Restrições

Esta secção apresenta as limitações conhecidas, condicionantes de utilização e avisos relevantes para a instalação, configuração e operação do **ScriptumAI**.

12.1 Limitações Funcionais

- **Inferência de IA em CPU:** O AI Service (Llama 3.1 via Ollama) necessita de alguma capacidade de GPU para desempenho adequado em produção.
- **Formatos de documento:** O processamento automático por IA suporta os formatos PDF, DOCX, TXT, PNG e JPG. Folhas de cálculo (XLSX) são armazenados mas não submetidos a extração de texto por OCR;
- **Idioma do chatbot RAG:** O assistente conversacional está otimizado para documentos em português e inglês. Documentos noutros idiomas podem produzir respostas com menor qualidade;

12.2 Segurança e Privacidade

- Os tokens JWT têm uma validade de 7 dias (configurável). Em caso de comprometimento de um token, este pode ser invalidado manualmente pelo Administrador através da *blacklist*;
- A plataforma implementa isolamento de dados por `organizationId`, mas a responsabilidade pela configuração correta dos subdomínios e pela gestão de utilizadores é do Administrador de cada organização.

13. Conclusão

13.1 Síntese

Este trabalho apresentou o desenvolvimento completo do **ScriptumAI**, uma plataforma *SaaS* multi-*tenant* para gestão inteligente de documentos, concebida para tornar capacidades de IA acessíveis a organizações de qualquer dimensão. A plataforma foi construída em três componentes complementares: um *backend* de microsserviços, uma aplicação web de administração e uma aplicação móvel em Android.

A arquitectura adoptada seguiu o padrão *Database per Service* com comunicação assíncrona via RabbitMQ, garantindo isolamento total entre *tenants* e modularidade evolutiva. O processamento documental assenta numa *pipeline* RAG (*Retrieval-Augmented Generation*) com o modelo Llama 3.1 e *embeddings* multilingues, permitindo que o assistente conversacional responda exclusivamente com base nos documentos de cada organização.

13.2 Resultados Principais

- **Backend:** cinco microsserviços independentes (Autenticação, Organização, Documento, IA e Notificações), cada um com base de dados PostgreSQL própria, orquestrados por Docker Compose em desenvolvimento e por Kubernetes em produção;
- **Aplicação web:** interface de administração completa com *dashboard* de KPIs, gestão de membros, convites e departamentos, secção de *staff* com controlo total sobre organizações e utilizadores, e suporte a modo *offline* via *service worker*;
- **Aplicação móvel:** aplicação Android com autenticação, gestão de documentos com filtros e ordenação, *chatbot* RAG por documento e global, notificações *push* via FCM e digitalização de documentos físicos via câmara (ML Kit);
- **Isolamento de dados:** os vectores ChromaDB e os documentos MinIO são particionados por organização, eliminando qualquer risco de acesso cruzado entre *tenants*;
- **Objetivos cumpridos:** a maioria dos objetivos funcionais propostos foi implementada; os casos não totalmente concluídos (planos de subscrição pagos) encontram-se identificados e justificados no capítulo de propósitos e objetivos cumpridos.

13.3 Limitações Observadas

- **Inferência de IA dependente de GPU:** o AI Service (Llama 3.1 via Ollama) necessita de alguma capacidade de GPU para desempenho adequado em produção; a execução em CPU é funcional mas significativamente mais lenta, tornando-a inadequada para volumes elevados de documentos;
- **Plano de subscrição único:** apenas o plano *Free Trial* foi implementado na *pipeline* de *onboarding*; os planos pagos estão modelados na base de dados mas a lógica de facturação e de imposição de limites por plano não está totalmente integrada;
- **Qualidade do OCR:** a extracção de texto em documentos digitalizados depende da qualidade da imagem; documentos com baixa resolução ou formatação complexa podem produzir *embeddings* de qualidade reduzida, afectando a relevância das respostas RAG;

Bibliografia

- Broadcom Inc. 2024. RabbitMQ documentation. <https://www.rabbitmq.com/documentation.html>.
- Chroma Research, Inc. 2024. ChromaDB — the open-source embedding database. <https://www.trychroma.com/>.
- Grafana Labs. 2024. k6 — load testing tool. <https://k6.io/docs/>.
- HashiCorp. 2024. Terraform documentation. <https://developer.hashicorp.com/terraform/docs>.
- Jones, M., J. Bradley & N. Sakimura. 2015. RFC 7519: JSON Web Token (JWT). <https://datatracker.ietf.org/doc/html/rfc7519>. Internet Engineering Task Force.
- LangChain, Inc. 2024. LangChain documentation. <https://python.langchain.com/>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel & Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems* 33. 9459–9474.
- NestJS Contributors. 2024. NestJS — a progressive Node.js framework. <https://nestjs.com/>.
- Ollama Contributors. 2024. Ollama — get up and running with large language models. <https://ollama.com/>.
- Parlamento Europeu e Conselho da União Europeia. 2016. Regulamento (ue) 2016/679 do parlamento europeu e do conselho (rgpd). <https://eur-lex.europa.eu/legal-content/PT/TXT/?uri=CELEX:32016R0679>.
- The Kubernetes Authors. 2024. Kubernetes documentation. <https://kubernetes.io/docs/>.
- Vercel, Inc. 2024. Next.js documentation. <https://nextjs.org/docs>.